

```
/* ===== 文件: src/app.config.ts ===== */
export default defineAppConfig({
  // 启用隐私保护框架（运行时 2.32.3+）：调用隐私接口（如 chooseAvatar）前需用户授权，
  // 配合 app.tsx 的 getPrivacySetting / auth.ts 的 ensurePrivacyAuthorized 在登录、发布前弹官方授权
  __usePrivacyCheck__: true,
  pages: [
    'pages/home/index',
    'pages/classics/index',
    'pages/insights/index',
    'pages/mine/index',
    'pages/settings/index',
    'pages/privacy/index'
  ],
  // 功能模块：章句详情、写笔记、译文搜索等次级页面 + 完整章节数据（按需下载，减小主模块体积）
  subPackages: [
    {
      root: 'packageContent',
      pages: [
        'pages/chapterDetail/index',
        'pages/verseDetail/index',
        'pages/writeNote/index',
        'pages/search/index'
      ]
    }
  ],
  // 功能模块预下载：首页/论语页是主入口，进入后预下载章句详情功能模块，消除首跳白屏
  preloadRule: {
    'pages/home/index': {
      network: 'all',
      packages: ['packageContent']
    },
    'pages/classics/index': {
      network: 'all',
      packages: ['packageContent']
    }
  },
  // 组件按需注入：移动端代码质量检查要求启用（应用开发平台"代码质量"未通过项）
  // 已知坑：开发者工具内 Taro 功能模块页面经主模块 base-view 引用主模块 comp 组件时，
  // 某些工具/运行时版本会白屏或报 file://not-found，真机正常（详见 docs/开发记录）。
  // 若在开发者工具复现，可切换调试运行时版本或升级开发者工具验证，真机发布不受影响。
  lazyCodeLoading: 'requiredComponents',
  window: {
    backgroundTextStyle: 'dark',
    navigationBarBackgroundColor: '#faf6f0',
    navigationBarTitleText: '论语学习',
    navigationBarTextStyle: 'black'
  },
  tabBar: {
    color: '#999999',
    selectedColor: '#b8612d',
```

```
backgroundColor: '#ffffff',
borderStyle: 'white',
list: [
  {
    pagePath: 'pages/home/index',
    text: '首页',
    iconPath: 'assets/tabbar/home.png',
    selectedIconPath: 'assets/tabbar/home-selected.png'
  },
  {
    pagePath: 'pages/classics/index',
    text: '论语',
    iconPath: 'assets/tabbar/classics.png',
    selectedIconPath: 'assets/tabbar/classics-selected.png'
  },
  {
    pagePath: 'pages/insights/index',
    text: '笔记',
    iconPath: 'assets/tabbar/insights.png',
    selectedIconPath: 'assets/tabbar/insights-selected.png'
  },
  {
    pagePath: 'pages/mine/index',
    text: '我的',
    iconPath: 'assets/tabbar/mine.png',
    selectedIconPath: 'assets/tabbar/mine-selected.png'
  }
]
})
/* ===== 文件: src/app.tsx ===== */
import { useEffect } from 'react';
import Taro from '@tarojs/taro';
import { CLOUD_ENV } from '@config/cloud';
import { silentLoginAndMerge } from '@services/auth';
// 全局样式
import './app.scss';
// 隐私合规（P1-5）：需要隐私授权时暂不静默登录/上传，等用户首次触发（我的页登录、发布等）时
// 通过 系统隐私授权接口 弹出系统授权框；无需授权或接口不可用时照旧静默登录
function privacyGateLogin(): void {
  if (process.env.TARO_ENV !== 'app' || !Taro.getPrivacySetting) {
    silentLoginAndMerge();
    return;
  }
  try {
    Taro.getPrivacySetting({
      success: (res) => {
        if (!res.needAuthorization) silentLoginAndMerge();
      },
    },
    fail: () => {
```

```
// 旧运行时/接口异常：保守放行，维持原行为
silentLoginAndMerge();
}
});
} catch (e) {
  console.warn('[App] 隐私授权检查失败（放行静默登录）:', e);
  silentLoginAndMerge();
}
}

function App(props) {
  // 初始化云开发环境（仅移动应用）
  useEffect(() => {
    if (process.env.TARO_ENV === 'app' && Taro.cloud) {
      try {
        // 显式指定云环境 ID，未配置时回退到默认环境
        // 注意：traceUser（用户行为统计）保持关闭——云开发 init 参数以首次调用为准，无法事后开启；
        // 隐私授权前不收集行为数据，关闭最稳妥（隐私政策页也无需声明行为统计）
        const initConfig: { env?: string } = {};
        if (CLOUD_ENV) {
          initConfig.env = CLOUD_ENV;
        }
        Taro.cloud.init(initConfig);
        console.log('[App] 云开发环境初始化成功', CLOUD_ENV ? `(env: ${CLOUD_ENV})` : '(默认环境)');
      } catch (e) {
        console.error('[App] 云开发环境初始化失败:', e);
      }
    }

    // 静默自动登录（openId 由平台注入，无需授权；失败静默，不影响使用）
    // 首次登录会自动合并匿名期间的学习进度到用户 key 并上传云端；需隐私授权时推迟到用户首次触发
    privacyGateLogin();

    // 注意：不在主模块做全量数据「空闲预热」——app.tsx 属主模块，静态引用 versesLoader
    // 会把 851KB 压缩 blob 打进主模块（实测 common.js 174KB → 1MB，第四轮审查确认），
    // 违背「完整数据进功能模块」的体积设计；首次进功能模块页面时解压（0.5-2s 低端机可接受）
  }, []);

  return props.children;
}

export default App;
/* ===== 文件: src/index.html ===== */
<!DOCTYPE html>
<html>
  <head>
    <meta content="text/html; charset=utf-8" http-equiv="Content-Type" />
    <meta name="viewport" content="width=375,user-scalable=no" />
    <meta name="apple-mobile-web-app-capable" content="yes" />
    <meta name="apple-touch-fullscreen" content="yes" />
    <meta name="format-detection" content="telephone=no,address=no" />
    <meta name="apple-mobile-web-app-status-bar-style" content="white" />
    <meta http-equiv="X-UA-Compatible" content="IE=edge,chrome=1" />
    <title>论语学习</title>
    <script>
```

```
// 固定 html fontSize, 防止 flexible 脚本动态修改
(function() {
  var html = document.documentElement;
  html.style.fontSize = '18px';
  Object.defineProperty(html.style, 'fontSize', {
    get: function() { return '18px'; },
    set: function() { /* 忽略所有修改 */ }
  });
})();
})();
<%= htmlWebpackPlugin.options.script %>
</script>
</head>
<body>
  <div id="app"></div>
</body>
</html>
/* ===== 文件: src/pages/home/index.tsx ===== */
import React, { useState, useMemo, useCallback } from 'react';
import { View, Text, ScrollView } from '@tarojs/components';
import Taro, { useDidShow, usePullDownRefresh } from '@tarojs/taro';
import styles from './index.module.scss';
import ProgressBar from '@components/ProgressBar';
import Skeleton from '@components/Skeleton';
import { versesIndex, type VerseIndex } from '@data/versesIndex';
import { chapters } from '@data/chapters';
import { getTodayRecommend } from '@data/dailyRecommend';
import { getProgress } from '@utils/storage';
const HomePage: React.FC = () => {
  const [readCount, setReadCount] = useState(0);
  const [totalReadDays, setTotalReadDays] = useState(1);
  const [noteCount, setNoteCount] = useState(() => getProgress().myNotes.length);
  const [readVersels, setReadVersels] = useState<number[]>([]);
  const [lastReadVersel, setLastReadVersel] = useState<number | undefined>(undefined);
  const [dailyRecommend, setDailyRecommend] = useState(() => getTodayRecommend());
  const refreshProgress = useCallback(() => {
    const progress = getProgress();
    setReadCount(progress.readVersels.length);
    setReadVersels(progress.readVersels);
    setLastReadVersel(progress.lastReadVersel);
    setTotalReadDays(progress.totalReadDays || 1);
    setNoteCount(progress.myNotes.length);
  }, []);
  useDidShow(() => {
    refreshProgress();
    // 每次进入首页轮换经典名句
    setClassicVerses(pickRandomVerses());
    // 每日推荐按天轮换: useState 只在挂载时算一次, 进页时重算, 避免挂后台过夜第二天仍是昨天的推荐
    setDailyRecommend(getTodayRecommend());
  });
  // 今日推荐: 只用轻量索引 (original 字段), 避免加载完整章节数据进主模块
```

```
// dailyVerse 由 dailyRecommend 同步派生（useMemo），避免 state+useEffect 导致首帧 null 闪烁
const dailyVerse = useMemo<VerseIndex | null>(
  () => versesIndex.find(v => v.id === dailyRecommend.verseId) || versesIndex[0] || null,
  [dailyRecommend]
);
// 下拉刷新：重新获取今日推荐与进度
usePullDownRefresh(async () => {
  setDailyRecommend(getTodayRecommend());
  refreshProgress();
  Taro.stopPullDownRefresh();
});
// 经典名句：随机取 5 条（每次进入页面轮换，避免老用户看腻固定内容）
const pickRandomVerses = useCallback(() : VerseIndex[] => {
  const shuffled = [...versesIndex].sort(() => Math.random() - 0.5);
  return shuffled.slice(0, 5);
}, []);
const [classicVerses, setClassicVerses] = useState<VerseIndex[]>(() => pickRandomVerses());
const totalVerses = useMemo(() => {
  return chapters.reduce((sum, c) => sum + c.verseCount, 0);
}, []);
const readChapters = useMemo(() => {
  // 先建 Set 再过滤（对齐 classics 页写法），避免 filter 内 includes 线性扫描 O(n×m)
  const readSet = new Set(readVerseIds);
  const readChapterIds = new Set(
    versesIndex.filter(v => readSet.has(v.id)).map(v => v.chapterId)
  );
  return readChapterIds.size;
}, [readVerseIds]);
// 接着读：最后阅读的章句（含篇章信息，仅用轻量索引）
const lastReadVerse = useMemo(() => {
  if (!lastReadVerseId) return null;
  const v = versesIndex.find(item => item.id === lastReadVerseId);
  if (!v) return null;
  const chapter = chapters.find(c => c.id === v.chapterId);
  return { ...v, chapterTitle: chapter?.title || '' };
}, [lastReadVerseId]);
const handleContinueRead = () => {
  if (!lastReadVerse) return;
  Taro.navigateTo({ url: `/packageContent/pages/verseDetail/index?id=${lastReadVerse.id}` });
};
const handleDailyClick = () => {
  if (!dailyVerse) return;
  Taro.navigateTo({ url: `/packageContent/pages/verseDetail/index?id=${dailyVerse.id}` });
};
const handleClassicClick = (verseId: number) => {
  Taro.navigateTo({ url: `/packageContent/pages/verseDetail/index?id=${verseId}` });
};
const handleNavigate = (path: string) => {
  Taro.switchTab({ url: path });
};
```

```
const handleMoreClassics = () => {
  Taro.switchTab({ url: '/pages/classics/index' });
};
// 搜索框入口：跳转功能模块搜索页（译文/注释全库搜索，含搜索历史）
const handleOpenSearch = () => {
  Taro.navigateTo({ url: '/packageContent/pages/search/index' });
};
return (
  <ScrollView className={styles.container} scrollY enhanced bounces>
    { /* 顶部标题 */ }
    <View className={styles.header}>
      <Text className={styles.appTitle}>论语学习</Text>
      <Text className={styles.appSubtitle}>学而时习之，不亦说乎</Text>
    </View>
    { /* 搜索入口（P2-1：最常驻页面提供搜索） */ }
    <View className={styles.searchBar} onClick={handleOpenSearch}>
      <Text className={styles.searchIcon}>搜</Text>
      <Text className={styles.searchPlaceholder}>搜索译文、注释...（支持全库搜索）</Text>
    </View>
    { /* 今日推荐 */ }
    <View className={styles.dailyCard} onClick={handleDailyClick}>
      <View className={styles.dailyLabel}>
        <Text className={styles.dailyLabelIcon}>日</Text>
        <Text className={styles.dailyLabelText}>今日推荐</Text>
        { /* 主题标签（方案 E：按主题轮换，20 篇 20 天一轮） */ }
        <Text className={styles.dailyTag}>{dailyRecommend.theme}</Text>
      </View>
      {dailyVerse ? (
        <Text className={styles.dailyOriginal}>{dailyVerse.original}</Text>
      ) : (
        { /* 骨架屏（P2-5）：数据未就绪时的加载占位 */ }
        <View className={styles.dailySkeleton}>
          <Skeleton height="44rpx" />
          <Skeleton height="44rpx" width="70%" style={{ marginTop: '16rpx' }} />
        </View>
      )}
      <Text className={styles.dailyChapter}>{dailyRecommend.chapterTitle}</Text>
      <View className={styles.dailyActions}>
        <View className={styles.dailyBtn} onClick={(e) => { e.stopPropagation(); handleDailyClick(
          <Text className={styles.dailyBtnText}>开始学习 </Text>
        </View>
      </View>
    </View>
    { /* 继续学习（上次阅读位置，点击续读） */ }
    {lastReadVerse && (
      <View className={styles.continueCard} onClick={handleContinueRead}>
        <View className={styles.dailyLabel}>
          <Text className={styles.dailyLabelIcon}>续</Text>
          <Text className={styles.dailyLabelText}>继续学习</Text>
        </View>
      </View>
    )}
  </ScrollView>
);
```

```
<Text className={styles.dailyOriginal} numberOfLines={2}>{lastReadVerse.original}</Text>
<View className={styles.dailyBtn}>
  <Text className={styles.dailyBtnText}>{lastReadVerse.chapterTitle} · {lastReadVerse.chap
</View>
</View>
)}
{/* 学习进度 */}
<View className={styles.progressSection}>
  <Text className={styles.progressTitle}>学习进度</Text>
  <View className={styles.statsRow}>
    <View className={styles.statItem}>
      <Text className={styles.statNumber}>{readCount}</Text>
      <Text className={styles.statLabel}>已读章句</Text>
    </View>
    <View className={styles.statItem}>
      <Text className={styles.statNumber}>{readChapters}</Text>
      <Text className={styles.statLabel}>已读篇目</Text>
    </View>
    <View className={styles.statItem}>
      <Text className={styles.statNumber}>{readCount === 0 ? 0 : totalReadDays} ☒</Text>
      <Text className={styles.statLabel}>连续学习</Text>
    </View>
    <View className={styles.statItem}>
      <Text className={styles.statNumber}>{noteCount}</Text>
      <Text className={styles.statLabel}>我的笔记</Text>
    </View>
  </View>
  <ProgressBar current={readCount} total={totalVerses} label="总进度" />
</View>
{/* 快捷入口 */}
<View className={styles.sectionTitle}>
  <Text className={styles.sectionTitleText}>快捷入口</Text>
</View>
<View className={styles.quickGrid}>
  <View className={styles.quickItem} onClick={() => handleNavigate('/pages/classics/index')}>
    <View className={styles.quickIcon}>
      <Text className={styles.quickIconText}>书</Text>
    </View>
    <View className={styles.quickInfo}>
      <Text className={styles.quickName}>篇章阅读</Text>
      <Text className={styles.quickDesc}>二十篇全文</Text>
    </View>
  </View>
  <View className={styles.quickItem} onClick={() => handleNavigate('/pages/insights/index')}>
    <View className={styles.quickIcon}>
      <Text className={styles.quickIconText}>记</Text>
    </View>
    <View className={styles.quickInfo}>
      <Text className={styles.quickName}>我的笔记</Text>
      <Text className={styles.quickDesc}>学习感悟记录</Text>
```

```
</View>
</View>
<View className={styles.quickItem} onClick={() => handleNavigate('/pages/mine/index')}}>
  <View className={styles.quickIcon}>
    <Text className={styles.quickIconText}>人</Text>
  </View>
  <View className={styles.quickInfo}>
    <Text className={styles.quickName}>个人中心</Text>
    <Text className={styles.quickDesc}>学习数据</Text>
  </View>
</View>
<View className={styles.quickItem} onClick={() => Taro.navigateTo({ url: '/pages/settings/in
  <View className={styles.quickIcon}>
    <Text className={styles.quickIconText}>设</Text>
  </View>
  <View className={styles.quickInfo}>
    <Text className={styles.quickName}>设置</Text>
    <Text className={styles.quickDesc}>数据管理</Text>
  </View>
</View>
</View>
{/* 经典名句 */}
<View className={styles.sectionTitle}>
  <Text className={styles.sectionTitleText}>经典名句</Text>
  <Text className={styles.sectionMore} onClick={handleMoreClassics}>查看全部 </Text>
</View>
<View className={styles.classicList}>
  {classicVerses.map(verse => (
    <View
      key={verse.id}
      className={styles.classicCard}
      onClick={() => handleClassicClick(verse.id)}
    >
      <Text className={styles.classicOriginal}>{verse.original}</Text>
    </View>
  ))}
</View>
</ScrollView>
);
};
export default HomePage;
/* ===== 文件: src/pages/classics/index.tsx ===== */
import React, { useState, useEffect, useMemo, useCallback } from 'react';
import { View, Text, Input, ScrollView } from '@tarojs/components';
import type { InputProps, BaseEventOrig } from '@tarojs/components';
import Taro, { useDidShow } from '@tarojs/taro';
import classNames from 'classnames';
import styles from './index.module.scss';
import ChapterCard from '@components/ChapterCard';
import { chapters } from '@data/chapters';
```



```
import { versesIndex } from '@data/versesIndex';
import { getProgress } from '@utils/storage';
import { useDebounce } from '@hooks/useDebounce';
import {
  searchInstant,
  getFieldLabel,
  type SearchResult
} from '@services/search';
import { renderHighlighted } from '@utils/highlight';
const PAGE_SIZE = 20; // 下滑加载每页条数
const ClassicsPage: React.FC = () => {
  const [searchText, setSearchText] = useState('');
  const [activeTheme, setActiveTheme] = useState('全部');
  const [readVersels, setReadVersels] = useState<number[]>([]);
  // 搜索框聚焦控制（P2-1：「搜原文」tab 点击后聚焦本页搜索框；
  // 移动端 Input 的 focus 需 false→true 切换才会重复聚焦，故点击时先关再开）
  const [searchFocus, setSearchFocus] = useState(false);
  // 即时搜索结果（原文，来自轻量索引）
  const [instantResults, setInstantResults] = useState<SearchResult[]>([]);
  // 已显示的搜索结果条数（下滑分页加载）
  const [displayCount, setDisplayCount] = useState(PAGE_SIZE);
  // 使用防抖优化搜索性能
  const debouncedSearchText = useDebounce(searchText, 300);
  useDidShow(() => {
    const progress = getProgress();
    setReadVersels(progress.readVersels);
  });
  // 即时搜索原文（本页始终为原文搜索；译文/注释搜索是独立入口，跳转功能模块页）
  useEffect(() => {
    const kw = debouncedSearchText.trim();
    if (!kw) {
      setInstantResults([]);
      return;
    }
    setInstantResults(searchInstant(kw));
    setDisplayCount(PAGE_SIZE); // 新搜索重置分页
  }, [debouncedSearchText]);
  // 点击显示全部搜索结果
  const handleShowAll = useCallback(() => {
    setDisplayCount(instantResults.length);
  }, [instantResults.length]);
  // 主题分类
  const themes = useMemo(() => {
    const uniqueThemes = Array.from(new Set(chapters.map(c => c.theme)));
    return ['全部', ...uniqueThemes];
  }, []);
  // 预构建篇章 id -> Chapter 映射（搜索结果渲染时避免反复 chapters.find）
  const chapterMap = useMemo(() => {
    const m = new Map<number, typeof chapters[number]>();
    chapters.forEach(c => m.set(c.id, c));
  }, []);
```

```
    return m;
  }, []);
// 预构建每篇已读句数（用 Set 加速 includes，避免每篇章全量扫描 versesIndex）
const readCountMap = useMemo(() => {
  const readSet = new Set(readVersesIds);
  const m = new Map<number, number>();
  for (const v of versesIndex) {
    if (readSet.has(v.id)) {
      m.set(v.chapterId, (m.get(v.chapterId) || 0) + 1);
    }
  }
  return m;
}, [readVersesIds]);
// 计算每篇已读句数（O(1) 查 Map）
const getReadCount = (chapterId: number): number => readCountMap.get(chapterId) || 0;
// 过滤篇章（按关键词和主题）
const filteredChapters = useMemo(() => {
  let result = chapters;
  if (activeTheme !== '全部') {
    result = result.filter(c => c.theme === activeTheme);
  }
  if (debouncedSearchText) {
    const text = debouncedSearchText.trim();
    result = result.filter(c =>
      c.title.includes(text) ||
      c.subTitle.includes(text) ||
      c.theme.includes(text) ||
      c.description.includes(text)
    );
  }
  return result;
}, [debouncedSearchText, activeTheme]);
const handleChapterClick = useCallback((chapterId: number) => {
  Taro.navigateTo({ url: `/packageContent/pages/chapterDetail/index?id=${chapterId}` });
}, []);
const handleVerseClick = useCallback((verseId: number) => {
  Taro.navigateTo({ url: `/packageContent/pages/verseDetail/index?id=${verseId}` });
}, []);
const handleSearchInput = useCallback((e: BaseEventOrig<InputProps.inputEventDetail>) => {
  setSearchText(e.detail.value);
}, []);
const handleClearSearch = useCallback(() => {
  setSearchText('');
}, []);
// 「搜原文」tab：聚焦本页搜索框（先置 false 再置 true，确保重复点击仍能聚焦）
const handleFocusSearch = useCallback(() => {
  setSearchFocus(false);
  setTimeout(() => setSearchFocus(true), 60);
}, []);
// 译文/注释搜索：独立入口，跳转功能模块搜索页（加载完整章节数据，不进主模块）
```

```
const handleOpenDeepSearch = useCallback(() => {
  const kw = searchText.trim();
  Taro.navigateTo({ url: `/packageContent/pages/search/index?keyword=${encodeURIComponent(kw)}` })
}, [searchText]);
const isSearching = debouncedSearchText.trim().length > 0;
const showChapterList = !isSearching || filteredChapters.length > 0;
// 原文搜索结果
const allResults = instantResults;
const totalVerseMatches = allResults.length;
return (
  <ScrollView className={styles.container} scrollY enhanced bounces>
    {/* 简介 */}
    <View className={styles.introCard}>
      <Text className={styles.introTitle}>论语二十篇</Text>
      <Text className={styles.introText}>儒家经典，孔子弟子及再传弟子编纂，记录孔子及其弟子言行，
    </View>
    {/* 搜索模式切换 */}
    <View className={styles.modeTabs}>
      <Text
        className={classnames(styles.modeTab, styles.modeTabActive)}
        onClick={handleFocusSearch}
      >
        搜原文
      </Text>
      <Text className={styles.modeTab} onClick={handleOpenDeepSearch}>搜译文/注释 </Text>
    </View>
    {/* 搜索栏 */}
    <View className={styles.searchBar}>
      <Text className={styles.searchIcon}>搜</Text>
      <Input
        className={styles.searchInput}
        placeholder="搜索原文..."
        value={searchText}
        onInput={handleSearchInput}
        confirmType="search"
        focus={searchFocus}
      />
      {isSearching && (
        <Text className={styles.clearBtn} onClick={handleClearSearch}></Text>
      )}
    </View>
    {/* 搜索结果概览 */}
    {isSearching && (
      <View className={styles.searchStats}>
        <Text className={styles.searchStatsText}>
          找到 {totalVerseMatches} 条章句、{filteredChapters.length} 篇篇章
        </Text>
      </View>
    )}
    {/* 搜索匹配的章句 */}
```

```
{isSearching && allResults.length > 0 && (
  <View className={styles.verseSection}>
    <View className={styles.sectionTitleRow}>
      <Text className={styles.sectionTitle}>匹配章句</Text>
      <Text className={styles.sectionCount}>共{totalVerseMatches}条</Text>
    </View>
    {allResults.slice(0, displayCount).map(result => {
      const chapter = chapterMap.get(result.chapterId);
      return (
        <View
          key={result.id}
          className={styles.searchResultCard}
          onClick={() => handleVerseClick(result.id)}
        >
          <View className={styles.resultHeader}>
            <Text className={styles.resultChapter}>{chapter?.title} · {result.chapterId}-{result.id}</Text>
            <View className={styles.matchBadges}>
              {result.matchFields.map(f => (
                <Text key={f} className={styles.matchBadge}>{getFieldLabel(f)}</Text>
              ))}
            </View>
          </View>
          <View>
            {/* 原文展示 */}
            <Text className={styles.resultOriginal} selectable>
              {renderHighlighted(result.original, debouncedSearchText, styles.highlight)}
            </Text>
            {/* 匹配上下文预览（非原文命中时展示） */}
            {result.previewField !== 'original' && (
              <Text className={styles.resultPreview} selectable>
                {renderHighlighted(result.previewText, debouncedSearchText, styles.highlight)}
              </Text>
            )}
          </View>
        </View>
      );
    })}
  </View>
  {/* 事件挂 View 而非 Text：避免 Taro 4.1.9 中 Text 节点复用导致 onClick 移除时
    读取不存在的 pure-text 别名而抛错（点击"显示全部"后切换为"已全部加载"场景） */}
  {allResults.length > PAGE_SIZE && (
    <View
      className={styles.resultMore}
      onClick={allResults.length > displayCount ? handleShowAll : undefined}
    >
      {allResults.length > displayCount
        ? `点击显示全部（共 ${allResults.length} 条，已显示 ${displayCount} 条）`
        : `已全部加载，共 ${allResults.length} 条`}
    </View>
  )}
</View>
)}
{/* 主题分类 */}
```

```

{!isSearching && (
  <ScrollView className={styles.tagScroll} scrollX enhanced showScrollbar={false}>
    {themes.map(theme => (
      <View
        key={theme}
        className={classnames(styles.tagItem, activeTheme === theme && styles.active)}
        onClick={() => setActiveTheme(theme)}
      >
        <Text>{theme}</Text>
      </View>
    ))}
  </ScrollView>
)}
{/* 篇章列表 */}
{showChapterList && (
  <>
    <View className={styles.listHeader}>
      <Text className={styles.listTitle}>{isSearching ? '匹配篇章' : '篇目列表'}</Text>
      <Text className={styles.listCount}>共 {filteredChapters.length} 篇</Text>
    </View>
    <View className={styles.chapterList}>
      {filteredChapters.map(chapter => (
        <ChapterCard
          key={chapter.id}
          chapter={chapter}
          readCount={getReadCount(chapter.id)}
          onClick={handleChapterClick}
        />
      ))}
    </View>
  </>
)}
{isSearching && filteredChapters.length === 0 && allResults.length === 0 && (
  <Text className={styles.emptyTip}>未找到匹配内容，试试其他关键词</Text>
)}
</ScrollView>
);
};

export default ClassicsPage;
/* ===== 文件: src/pages/insights/index.tsx ===== */
import React, { useState, useCallbak } from 'react';
import { View, Text, ScrollView } from '@tarojs/components';
import Taro, { useDidShow } from '@tarojs/taro';
import styles from './index.module.scss';
import { getProgress, deleteNote } from '@/utils/storage';
import { versesIndex } from '@/data/versesIndex';
import type { MyNote } from '@/types';
// 我的笔记（本地笔记列表，去 UGC 后承载原「心得」tab）
const InsightsPage: React.FC = () => {
  const [myNotes, setMyNotes] = useState<MyNote[]>(() => getProgress().myNotes);

```

```
useDidShow(() => {
  // 每次进入刷新（写笔记/编辑后返回时同步最新列表）
  setMyNotes(getProgress().myNotes);
});
const handleNoteClick = (verseId: number) => {
  Taro.navigateTo({ url: `/packageContent/pages/verseDetail/index?id=${verseId}` });
};
// 编辑笔记
const handleEditNote = useCallback((note: MyNote) => {
  Taro.navigateTo({ url: `/packageContent/pages/writeNote/index?verseId=${note.verseId}&noteId=${note.id}` }, []);
// 删除笔记
const handleDeleteNote = useCallback((note: MyNote) => {
  Taro.showModal({
    title: '删除笔记',
    content: '确定要删除这条笔记吗？删除后不可恢复。',
    confirmColor: '#B8612D',
    success: (res) => {
      if (res.confirm) {
        try {
          deleteNote(note.id);
          Taro.showToast({ title: '已删除', icon: 'success' });
          setMyNotes(getProgress().myNotes);
        } catch (e: any) {
          Taro.showToast({ title: e?.message || '删除失败', icon: 'none' });
        }
      }
    }
  });
}, []);
return (
  <ScrollView className={styles.container} scrollY enhanced bounces>
    <View className={styles.header}>
      <Text className={styles.title}>我的笔记</Text>
      <Text className={styles.subtitle}>记录你的学习感悟</Text>
    </View>
    {myNotes.length > 0 ? (
      myNotes.map(note => {
        const verse = versesIndex.find(v => v.id === note.verseId);
        return (
          <View
            key={note.id}
            className={styles.noteCard}
            onClick={() => handleNoteClick(note.verseId)}
          >
            <Text className={styles.noteContent}>{note.content}</Text>
            {note.tags && note.tags.length > 0 && (
              <View className={styles.noteTags}>
                {note.tags.map((tag, idx) => (
                  <Text key={`_${tag}-${idx}`} className={styles.noteTag}>{tag}</Text>
                ))}
              </View>
            )}
          </View>
        );
      })
    ) : null}
  </ScrollView>
);
```

```
    ))}
  </View>
)}
<View className={styles.noteFooter}>
  <Text className={styles.noteTime}>{note.createTime}</Text>
  {verse && <Text className={styles.noteSource}>出自: {verse.original.substring(0, 12)}
</View>
<View className={styles.noteActions}>
  <Text className={styles.noteAction} onClick={(e) => { e.stopPropagation(); handleEdi
  <Text className={styles.noteActionDelete} onClick={(e) => { e.stopPropagation(); han
  </View>
</View>
);
})
): (
  <View className={styles.emptyNotes}>
    <Text>还没有笔记，去学习后写第一条笔记吧</Text>
  </View>
)}
</ScrollView>
);
};
export default InsightsPage;
/* ===== 文件: src/pages/mine/index.tsx ===== */
import React, { useState, useMemo, useCallback } from 'react';
import { View, Text, ScrollView, Image, Button, Input } from '@tarojs/components';
import Taro, { useDidShow } from '@tarojs/taro';
import classNames from 'classnames';
import styles from './index.module.scss';
import ProgressBar from '@components/ProgressBar';
import { getProgress } from '@utils/storage';
import { versesIndex } from '@data/versesIndex';
import { chapters } from '@data/chapters';
import { getUserInfo, silentLoginAndMerge, logout, updateProfile, ensurePrivacyAuthorized } from '@
import type { UserInfo } from '@types';
const MinePage: React.FC = () => {
  const [progress, setProgress] = useState(getProgress());
  const [user, setUser] = useState<UserInfo | null>(getUserInfo());
  const [logging, setLogging] = useState(false);
  const [avatarFailed, setAvatarFailed] = useState(false);
  // 资料编辑弹层
  const [showProfileEditor, setShowProfileEditor] = useState(false);
  const [editNickName, setEditNickName] = useState('');
  const [editAvatarUrl, setEditAvatarUrl] = useState('');
  const [savingProfile, setSavingProfile] = useState(false);
  // 打开资料编辑（预填当前昵称/头像）
  const openProfileEditor = () => {
    setEditNickName(user?.nickName && user.nickName !== '论语学习者' ? user.nickName : '');
    setEditAvatarUrl(user?.avatarUrl || '');
    setShowProfileEditor(true);
  }
}
```

```
};  
// 保存资料（云函数 updateProfile + 本地缓存）  
const handleSaveProfile = async () => {  
  const nickName = editNickName.trim() || '论语学习者';  
  if (savingProfile) return;  
  setSavingProfile(true);  
  try {  
    const u = await updateProfile({ nickName, avatarUrl: editAvatarUrl });  
    setUser(u);  
    setAvatarFailed(false);  
    setShowProfileEditor(false);  
    Taro.showToast({ title: '资料已更新', icon: 'success' });  
  } catch (e: any) {  
    console.error('[Mine] 更新资料失败:', e);  
    Taro.showToast({ title: e?.message || '更新失败，请重试', icon: 'none' });  
  } finally {  
    setSavingProfile(false);  
  }  
};  
  
useDidShow(() => {  
  setProgress(getProgress());  
  const u = getUserInfo();  
  setUser(u);  
  // 用户信息刷新（换头像等）时重置头像加载失败标记  
  setAvatarFailed(false);  
});  
  
const readCount = progress.readVerselds.length;  
const totalReadDays = progress.totalReadDays || 1;  
const totalVerses = useMemo(() => chapters.reduce((sum, c) => sum + c.verseCount, 0), []);  
const readChapterIds = useMemo(() => {  
  return new Set(  
    versesIndex.filter(v => progress.readVerselds.includes(v.id)).map(v => v.chapterId)  
  );  
}, [progress]);  
  
const loggedIn = user !== null;  
// 登录（App 启动已静默自动登录；此处按钮保留兜底：登录 + 合并数据）  
const handleLogin = useCallback(async () => {  
  if (logging) return;  
  setLogging(true);  
  try {  
    // 隐私合规：登录（云同步数据上传）前先确保用户同意隐私政策，未授权则弹官方授权框  
    const authorized = await ensurePrivacyAuthorized();  
    if (!authorized) {  
      Taro.showToast({ title: '需同意隐私政策后才能登录', icon: 'none' });  
      return;  
    }  
    const u = await silentLoginAndMerge();  
    if (!u) throw new Error('登录失败，请重试');  
    setUser(u);  
    setProgress(getProgress());  
  }  
};
```



```
Taro.showToast({ title: '登录成功', icon: 'success' });
} catch (e: any) {
  console.error('[Mine] login failed:', e);
  Taro.showToast({ title: e?.message || '登录失败', icon: 'none' });
} finally {
  setLogging(false);
}
}, [logging]);
// 退出登录
const handleLogout = useCallback(() => {
  Taro.showModal({
    title: '退出登录',
    content: '退出后本地数据仍会保留，确定退出吗？',
    success: (res) => {
      if (res.confirm) {
        logout();
        setUser(null);
        setProgress(getProgress());
        Taro.showToast({ title: '已退出登录', icon: 'none' });
      }
    }
  });
}, []);
const handleMenuClick = (type: string) => {
  if (type === 'continue') {
    // 真正续读：有上次阅读位置直接进章句详情，否则去论语列表
    const lastReadVerselId = getProgress().lastReadVerselId;
    if (lastReadVerselId) {
      Taro.navigateTo({ url: `/packageContent/pages/verseDetail/index?id=${lastReadVerselId}` });
    } else {
      Taro.switchTab({ url: '/pages/classics/index' });
    }
  } else if (type === 'classics') {
    Taro.switchTab({ url: '/pages/classics/index' });
  } else if (type === 'insights') {
    // 我的笔记：内容已移至「心得」tab
    Taro.switchTab({ url: '/pages/insights/index' });
  } else if (type === 'settings') {
    Taro.navigateTo({ url: '/pages/settings/index' });
  }
};
return (
  <ScrollView className={styles.container} scrollY enhanced bounces>
    { /* 用户卡片 */ }
    <View className={styles.userCard}>
      {loggedIn && user ? (
        <View className={styles.userInfo}>
          <View className={styles.avatar}>
            {user.avatarUrl && !avatarFailed ? (
              <Image
```

```

        className={styles.avatarImg}
        src={user.avatarUrl}
        mode="aspectFill"
        lazyLoad
        onError={() => setAvatarFailed(true)}
    />
) : (
    <Text className={styles.avatarText}>
        {(user.nickname || '学').charAt(0)}
    </Text>
    <View>
        <View className={styles.userName}>
            <Text className={styles.userName}>{user.nickname || '论语学习者'}</Text>
            <Text className={styles.userDesc} onClick={openProfileEditor}>学而时习之，不亦说乎
        </View>
        <View className={styles.logoutBtn} onClick={handleLogout}>
            <Text className={styles.logoutText}>退出</Text>
        </View>
    </View>
) : (
    <View className={styles.loginArea} onClick={handleLogin}>
        <View className={styles.avatar}>
            <Text className={styles.avatarText}>学</Text>
        </View>
        <View className={styles.userName}>
            <Text className={styles.userName}>
                {logging ? '登录中...' : '点击登录'}
            </Text>
            <Text className={styles.userDesc}>登录后同步学习进度到云端</Text>
        </View>
        <View className={styles.loginBtn}>
            <Text className={styles.loginBtnText}>登录 </Text>
        </View>
    </View>
    <View>
        <View className={styles.statsCard}>
            <View className={styles.statsRow}>
                <View className={styles.statItem}>
                    <Text className={styles.statNumber}>{readCount}</Text>
                    <Text className={styles.statLabel}>已读章句</Text>
                </View>
                <View className={styles.statItem}>
                    <Text className={styles.statNumber}>{readChapterIds.size}</Text>
                    <Text className={styles.statLabel}>已读篇目</Text>
                </View>
                <View className={styles.statItem}>
                    <Text className={styles.statNumber}>{readCount === 0 ? 0 : totalReadDays} </Text>

```

```
<Text className={styles.statLabel}>连续学习</Text>
</View>
<View className={styles.statItem}>
  <Text className={styles.statNumber}>{progress.myNotes.length}</Text>
  <Text className={styles.statLabel}>我的笔记</Text>
</View>
</View>
</View>
{/* 学习进度 */}
<View className={styles.progressSection}>
  <Text className={styles.progressTitle}>学习总进度</Text>
  <ProgressBar current={readCount} total={totalVerses} label="论语二十篇" />
</View>
{/* 功能菜单 */}
<View className={styles.menuSection}>
  <View className={styles.menuItem} onClick={() => handleMenuClick('continue')}>
    <View className={styles.menuIcon}>
      <Text className={styles.menuIconText}>读</Text>
    </View>
    <Text className={styles.menuText}>继续阅读</Text>
    <Text className={styles.menuArrow}>☞</Text>
  </View>
  <View className={styles.menuItem} onClick={() => handleMenuClick('insights')}>
    <View className={styles.menuIcon}>
      <Text className={styles.menuIconText}>记</Text>
    </View>
    <Text className={styles.menuText}>我的笔记</Text>
    <Text className={styles.menuArrow}>☞</Text>
  </View>
  <View className={styles.menuItem} onClick={() => handleMenuClick('settings')}>
    <View className={styles.menuIcon}>
      <Text className={styles.menuIconText}>置</Text>
    </View>
    <Text className={styles.menuText}>设置</Text>
    <Text className={styles.menuArrow}>☞</Text>
  </View>
</View>
{/* 资料编辑弹层（头像昵称填写能力：chooseAvatar 按钮 + nickname 输入框） */}
{showProfileEditor && (
  <View className={styles.profileMask} onClick={() => setShowProfileEditor(false)}>
    <View className={styles.profilePanel} onClick={(e) => e.stopPropagation()}>
      <Text className={styles.profileTitle}>编辑资料</Text>
      {/* 头像选择：系统头像昵称填写能力 */}
      {/* chooseAvatar 返回临时路径（file:///tmp_*），会被系统随时清理，须先上传云存储拿 fileId */}
      <Button
        className={styles.avatarPicker}
        openType="chooseAvatar"
        onChooseAvatar={async (e) => {
          const tempPath = e.detail.avatarUrl;
          if (!tempPath) return;
        }}
      />
    </View>
  </View>
)
```

```
    if (process.env.TARO_ENV !== 'app') {
      // 非应用环境（H5 调试）无云存储，直接使用临时路径
      setEditAvatarUrl(tempPath);
      return;
    }
    try {
      const cloudPath = `avatars/${Date.now()}-${Math.random().toString(36).slice(2, 8)}`
      const res = await Taro.cloud.uploadFile({ cloudPath, filePath: tempPath });
      setEditAvatarUrl(res.fileID);
    } catch (err) {
      console.error('[Mine] 头像上传失败:', err);
      Taro.showToast({ title: '头像上传失败，请重试', icon: 'none' });
    }
  }}
>
{editAvatarUrl ? (
  <Image className={styles.avatarPickerImg} src={editAvatarUrl} mode="aspectFill" />
) : (
  <Text className={styles.avatarPickerText}>选择头像</Text>
)}
</Button>
{/* 昵称输入：常用昵称联想输入 */}
<Input
  className={styles.nicknameInput}
  type="nickname"
  placeholder="输入昵称（可联想常用昵称）"
  value={editNickName}
  maxLength={20}
  onInput={(e) => setEditNickName(e.detail.value)}
/>
<View className={styles.profileActions}>
  <View className={styles.profileCancel} onClick={() => setShowProfileEditor(false)}>
    <Text>取消</Text>
  </View>
  <View
    className={classnames(styles.profileSave, savingProfile && styles.profileSaveDisable)}
    onClick={handleSaveProfile}
  >
    <Text>{savingProfile ? '保存中...' : '保存'}</Text>
  </View>
</View>
</View>
</ScrollView>
);
};
export default MinePage;
/* ===== 文件: src/pages/settings/index.tsx ===== */
import React, { useState, useCallback } from 'react';
```

```
import { View, Text, ScrollView, Switch } from '@tarojs/components';
import Taro, { useDidShow } from '@tarojs/taro';
import classNames from 'classnames';
import styles from './index.module.scss';
import { saveProgress, clearPendingSync } from '@/utils/storage';
import { getSettings, saveSettings, type AppSettings, type FontSize } from '@/utils/settings';
import { syncProgressNow } from '@/services/sync';
import { isLoggedIn, getUserInfo, clearCloudProgress, ensurePrivacyAuthorized } from '@/services/auth';
import type { UserInfo } from '@/types';

const FONT_OPTIONS: { value: FontSize; label: string }[] = [
  { value: 'normal', label: '标准' },
  { value: 'large', label: '大' },
  { value: 'xl', label: '特大' }
];

// 数据来源：和合文化屋内容站点（文章专辑链接，应用内无法直接打开外链，采用复制方式提供）
const DATA_SOURCE_URL = 'https://data.lunyu-app.cn/album';

const SettingsPage: React.FC = () => {
  const [settings, setSettings] = useState<AppSettings>(() => getSettings());
  const [user, setUser] = useState<UserInfo | null>(() => getUserInfo());
  const [showClearConfirm, setShowClearConfirm] = useState(false);
  // 从设置页返回时同步最新状态
  useDidShow(() => {
    setSettings(getSettings());
    setUser(getUserInfo());
  });
  const applySettings = useCallback((patch: Partial<AppSettings>) => {
    const next = { ...getSettings(), ...patch };
    saveSettings(next);
    setSettings(next);
  }, []);
  const goLogin = () => {
    Taro.switchTab({ url: '/pages/mine/index' });
  };
  // 手动双向同步（下载 → 合并写回本地 → 上传）
  const handleSync = useCallback(async () => {
    if (!isLoggedIn()) {
      Taro.showToast({ title: '请先登录', icon: 'none' });
      return;
    }
    Taro.showLoading({ title: '同步中...' });
    try {
      const res = await syncProgressNow();
      Taro.hideLoading();
      Taro.showToast({ title: res.message, icon: res.success ? 'success' : 'none' });
    } catch (e) {
      Taro.hideLoading();
      Taro.showToast({ title: '同步失败', icon: 'none' });
    }
  }, []);
  const handleClearData = () => {
```

```
setShowClearConfirm(true);
};

const confirmClear = useCallback(async () => {
  setShowClearConfirm(false);
  // 清空本地学习数据（应用设置不受影响）
  saveProgress({
    readVerselds: [],
    myNotes: [],
    totalReadDays: 1,
    lastReadDate: undefined,
    lastReadVerseld: undefined,
    deletedNoteIds: [],
    likedNoteIds: [],
    unlikedNoteIds: []
  });
  // 取消 saveProgress 触发的 3 秒防抖上传，避免「清空云端」意图与自动上传时序冲突
  clearPendingSync();
  let message = '数据已清空';
  let icon: 'success' | 'none' = 'success';
  // 已登录则同步清空云端，避免旧数据在下次同步时复活
  if (isLoggedIn()) {
    try {
      const res = await clearCloudProgress();
      if (!res.success) {
        message = '本地已清空，云端清理失败';
        icon = 'none';
      }
    } catch (e) {
      message = '本地已清空，云端清理失败';
      icon = 'none';
    }
  }
  Taro.showToast({ title: message, icon, duration: 2000 });
  setTimeout(() => {
    // 用户可能已手动返回，栈深不足时不重复导航
    if (Taro.getCurrentPages().length > 1) {
      Taro.navigateBack();
    }
  }, 2000);
}, []);

const cancelClear = () => {
  setShowClearConfirm(false);
};

const handlePrivacy = () => {
  Taro.navigateTo({ url: '/pages/privacy/index' });
};

const handleAbout = () => {
  Taro.showModal({
    title: '关于论语学习',
    content: '版本: 1.0.0\n\n一款专注于《论语》学习的移动应用，提供原文、译文、注释解读及学习笔记'
```

```

    showCancel: false,
    confirmText: '知道了'
  });
};

// 复制数据来源链接到剪贴板（应用无法直接打开内容站点外链）
// setClipboardData 属隐私接口：先确保用户已同意隐私协议（未声明/未授权会 errno 112 失败）
// 注意：须在管理后台「用户隐私保护指引」声明「剪贴板（用于复制数据来源链接）」，否则走 fail 兜底
const handleDataSource = async () => {
  const authorized = await ensurePrivacyAuthorized();
  if (!authorized) {
    Taro.showToast({ title: '需同意隐私政策后才能复制', icon: 'none' });
    return;
  }
  Taro.setClipboardData({
    data: DATA_SOURCE_URL,
    success: () => {
      Taro.showToast({ title: '链接已复制', icon: 'success' });
    },
    fail: () => {
      // 后台隐私指引未声明剪贴板 scope 时移动端直接拒绝（errno 112），静默提示不崩溃
      Taro.showToast({ title: '复制失败，请稍后重试', icon: 'none' });
    }
  });
};

return (
  <ScrollView className={styles.container} scrollY enhanced bounces>
    {/* 页面标题 */}
    <View className={styles.header}>
      <Text className={styles.title}>设置</Text>
    </View>
    {/* 账号与同步 */}
    <View className={styles.section}>
      <Text className={styles.sectionTitle}>账号与同步</Text>
      <View className={styles.menuItem}>
        <View className={styles.menuIcon}>
          <Text className={styles.menuIconText}>{user ? (user.nickName || '学').charAt(0) : '未'}</Text>
        </View>
        <View className={styles.menuInfo}>
          <Text className={styles.menuText}>{user ? (user.nickName || '论语学习者') : '未登录'}</Text>
          <Text className={styles.menuDesc}>{user ? '已开启云端同步' : '登录后同步学习进度到云端'}</Text>
        </View>
        {!user && (
          <Text className={styles.menuAction} onClick={goLogin}>去登录</Text>
        )}
      </View>
      <View className={styles.menuItem} onClick={handleSync}>
        <View className={styles.menuIcon}>
          <Text className={styles.menuIconText}>同</Text>
        </View>
        <View className={styles.menuInfo}>

```

```
<Text className={styles.menuText}>同步学习进度</Text>
<Text className={styles.menuDesc}>下载并上传合并云端数据</Text>
</View>
<Text className={styles.menuArrow}><img alt="arrow icon" data-bbox="455 105 475 115"/></Text>
</View>
<View className={styles.menuItem}>
  <View className={styles.menuIcon}>
    <Text className={styles.menuIconText}>自</Text>
  </View>
  <View className={styles.menuInfo}>
    <Text className={styles.menuText}>自动同步</Text>
    <Text className={styles.menuDesc}>保存进度后自动上传云端</Text>
  </View>
  <Switch
    checked={settings.autoSync}
    onChange={(e) => applySettings({ autoSync: e.detail.value })}
    color="#b8612d"
  />
</View>
</View>
{ /* 阅读体验 */ }
<View className={styles.section}>
  <Text className={styles.sectionTitle}>阅读体验</Text>
  <View className={styles.menuItem}>
    <View className={styles.menuIcon}>
      <Text className={styles.menuIconText}>字</Text>
    </View>
    <View className={styles.menuInfo}>
      <Text className={styles.menuText}>正文字号</Text>
      <Text className={styles.menuDesc}>影响原文、译文与注释</Text>
    </View>
    <View className={styles.fontOptions}>
      {FONT_OPTIONS.map(opt => (
        <Text
          key={opt.value}
          className={classnames(styles.fontOption, settings.fontSize === opt.value && styles.f
            onClick={() => applySettings({ fontSize: opt.value })}
        >
          {opt.label}
        </Text>
      ))}
    </View>
  </View>
</View>
{ /* 隐私 */ }
<View className={styles.section}>
  <Text className={styles.sectionTitle}>隐私</Text>
  <View className={styles.menuItem} onClick={handlePrivacy}>
    <View className={styles.menuIcon}>
      <Text className={styles.menuIconText}>隐</Text>
```



```
</View>
<View className={styles.menuInfo}>
  <Text className={styles.menuText}>隐私政策</Text>
  <Text className={styles.menuDesc}>了解我们如何收集和使用数据</Text>
</View>
<Text className={styles.menuArrow}>☐/Text>
</View>
</View>
{/* 数据管理 */}
<View className={styles.section}>
  <Text className={styles.sectionTitle}>数据管理</Text>
  <View className={styles.menuItem} onClick={handleClearData}>
    <View className={styles.menuIcon}>
      <Text className={styles.menuIconText}>清</Text>
    </View>
    <View className={styles.menuInfo}>
      <Text className={styles.menuText}>清空学习数据</Text>
      <Text className={styles.menuDesc}>已读记录与本地笔记</Text>
    </View>
    <Text className={styles.menuArrow}>☐/Text>
  </View>
</View>
</View>
{/* 关于 */}
<View className={styles.section}>
  <Text className={styles.sectionTitle}>关于</Text>
  <View className={styles.menuItem} onClick={handleDataSource}>
    <View className={styles.menuIcon}>
      <Text className={styles.menuIconText}>源</Text>
    </View>
    <View className={styles.menuInfo}>
      <Text className={styles.menuText}>数据来源</Text>
      <Text className={styles.menuDesc}>和合文化屋内容站点</Text>
    </View>
    <Text className={styles.menuArrow}>☐/Text>
  </View>
  <View className={styles.menuItem} onClick={handleAbout}>
    <View className={styles.menuIcon}>
      <Text className={styles.menuIconText}>关</Text>
    </View>
    <View className={styles.menuInfo}>
      <Text className={styles.menuText}>关于论语学习</Text>
      <Text className={styles.menuDesc}>版本与数据来源</Text>
    </View>
    <Text className={styles.menuArrow}>☐/Text>
  </View>
</View>
</View>
{/* 版本信息 */}
<View className={styles.versionInfo}>
  <Text className={styles.versionText}>论语学习 v1.0.0</Text>
  <Text className={styles.copyright}>☐2026 论语学习团队</Text>
```

```
</View>
{/* 清空确认弹窗 */}
{showClearConfirm && (
  <View className={styles.modalOverlay} onClick={cancelClear}>
    <View className={styles.modal} onClick={(e) => e.stopPropagation()}>
      <Text className={styles.modalTitle}>确认清空</Text>
      <Text className={styles.modalContent}>
        将清空已读记录与本地笔记，且无法恢复。确定继续吗？
      </Text>
      <View className={styles.modalActions}>
        <View className={styles.modalBtnCancel} onClick={cancelClear}>
          <Text>取消</Text>
        </View>
        <View className={styles.modalBtnConfirm} onClick={confirmClear}>
          <Text>确认清空</Text>
        </View>
      </View>
    </View>
  )}
</ScrollView>
);
};

export default SettingsPage;
/* ===== 文件: src/pages/privacy/index.tsx ===== */
import React from 'react';
import { View, Text, ScrollView } from '@tarojs/components';
import Taro from '@tarojs/taro';
import styles from './index.module.scss';

const PrivacyPage: React.FC = () => {
  // 打开移动端官方《应用隐私保护指引》（需在应用管理后台配置隐私保护指引后可用）
  const handleOpenContract = () => {
    Taro.openPrivacyContract({
      fail: () => {
        Taro.showToast({ title: '暂不可用，请稍后再试', icon: 'none' });
      }
    });
  };

  return (
    <ScrollView className={styles.container} scrollY enhanced bounces>
      <View className={styles.section}>
        <Text className={styles.h2}>隐私政策</Text>
        <Text className={styles.p}>更新日期：2026-08-14</Text>
        <Text className={styles.p}>
          本应用（「论语学习」）由论语学习团队提供。我们重视你的隐私，本政策说明我们收集和使用哪些信
        </Text>
      </View>
      <View className={styles.section}>
        <Text className={styles.h3}>1. 我们收集的信息</Text>
        <Text className={styles.p}>- 学习进度（已读章句、笔记），默认仅保存在你的设备本地；</Text>
      </View>
    </ScrollView>
  );
};
```

```

    <Text className={styles.p}>- 用户登录信息（openId、昵称、头像），用于云端同步学习进度与笔记
    <Text className={styles.p}>- 剪贴板：在「设置 → 数据来源」复制链接时使用（仅在你点击复制时
</View>
<View className={styles.section}>
    <Text className={styles.h3}>2. 信息的使用</Text>
    <Text className={styles.p}>- 本地数据仅用于在你设备上展示学习进度与笔记；</Text>
    <Text className={styles.p}>- 云端数据用于多设备同步；</Text>
    <Text className={styles.p}>- 我们不会将你的个人信息出售给任何第三方。</Text>
</View>
<View className={styles.section}>
    <Text className={styles.h3}>3. 云同步与用户行为统计</Text>
    <Text className={styles.p}>
        - 云同步：登录后，你的学习进度与本地笔记会上传至云端，仅用于多设备同步。在你同意隐私授权之
    </Text>
    <Text className={styles.p}>
        - 用户行为统计：未征得你的同意前，我们不会开启行为数据统计。你首次登录时，系统会弹出系统隐
    </Text>
</View>
<View className={styles.section}>
    <Text className={styles.h3}>4. 你的权利</Text>
    <Text className={styles.p}>
        你可以在「设置 → 清空学习数据」清除本地学习数据。
    </Text>
</View>
<View className={styles.section}>
    <Text className={styles.h3}>5. 移动端官方隐私保护指引</Text>
    <Text className={styles.p}>
        除本政策外，应用平台对应用还有统一的隐私保护要求，可点击下方按钮查看官方指引。
    </Text>
    <View className={styles.contractBtn} onClick={handleOpenContract}>
        <Text>查看《移动应用隐私保护指引》</Text>
    </View>
</View>
<View className={styles.section}>
    <Text className={styles.h3}>6. 联系我们</Text>
    <Text className={styles.p}>
        如有隐私相关问题，可通过移动应用客服或意见反馈渠道联系我们（联系方式将在上线前补充）。
    </Text>
</View>
</ScrollView>
);
};
export default PrivacyPage;
/* ===== 文件：src/packageContent/pages/chapterDetail/index.tsx ===== */
import React, { useState, useMemo, useEffect } from 'react';
import { View, Text, ScrollView } from '@tarojs/components';
import Taro, { useRouter, useDidShow } from '@tarojs/taro';
import styles from './index.module.scss';
import VerseCard from '@components/VerseCard';
import ProgressBar from '@components/ProgressBar';

```

```
import { chapters } from '@/data/chapters';
import { loadChapter } from '@/data/versesLoader';
import type { Verse } from '@/types';
import { getProgress } from '@/utils/storage';
const ChapterDetailPage: React.FC = () => {
  const router = useRouter();
  const rawChapterId = Number(router.params.id || '1');
  // 无效 id 回退到第 1 篇，保证头部与列表一致
  const chapterId = chapters.some(c => c.id === rawChapterId) ? rawChapterId : 1;
  const [readVersels, setReadVersels] = useState<number[]>([]);
  const [noteVersels, setNoteVersels] = useState<number[]>([]);
  useDidShow(() => {
    const progress = getProgress();
    setReadVersels(progress.readVersels);
    setNoteVersels(progress.myNotes.map(n => n.verselId));
  });
  const chapter = useMemo(() => {
    return chapters.find(c => c.id === chapterId) || chapters[0];
  }, [chapterId]);
  const [chapterVerses, setChapterVerses] = useState<Verse[]>([]);
  const [loading, setLoading] = useState(true);
  const [loadFailed, setLoadFailed] = useState(false);
  const [retryTick, setRetryTick] = useState(0); // 重试计数，变化时重新触发加载
  useEffect(() => {
    let cancelled = false;
    setLoadFailed(false);
    (async () => {
      try {
        const verses = await loadChapter(chapterId);
        const sorted = [...verses].sort((a, b) => a.order - b.order);
        if (!cancelled) {
          setChapterVerses(sorted);
          setLoading(false);
        }
      } catch (e) {
        console.error('[ChapterDetail] Load chapter failed:', e);
        if (!cancelled) {
          setLoading(false);
          setLoadFailed(true);
          Taro.showToast({ title: '加载失败，请重试', icon: 'none', duration: 2000 });
        }
      }
    })();
    return () => { cancelled = true; };
  }, [chapterId, retryTick]);
  const readCount = useMemo(() => {
    return chapterVerses.filter(v => readVersels.includes(v.id)).length;
  }, [chapterVerses, readVersels]);
  const handleVerseClick = (verselId: number) => {
    Taro.navigateTo({ url: `/packageContent/pages/verseDetail/index?id=${verselId}` });
  };
};
```

```
};
return (
  <ScrollView className={styles.container} scrollY enhanced bounces>
    {/* 篇章头部 */}
    <View className={styles.chapterHeader}>
      <Text className={styles.chapterTitle}>{chapter.title}</Text>
      <Text className={styles.chapterTheme}>{chapter.theme}</Text>
      <Text className={styles.chapterDesc}>{chapter.description}</Text>
    </View>
    {/* 进度 */}
    {chapterVerses.length > 0 && (
      <ProgressBar current={readCount} total={chapterVerses.length} label="本篇进度" />
    )}
    {/* 句子列表 */}
    <View className={styles.listTitle}>
      <Text className={styles.listTitleText}>章句列表</Text>
      <Text className={styles.listCount}>共 {chapterVerses.length} 章</Text>
    </View>
    <View className={styles.verseList}>
      {loading ? (
        <Text className={styles.emptyTip}>加载中...</Text>
      ) : loadFailed ? (
        <View className={styles.loadFailedWrap}>
          <Text className={styles.emptyTip}>加载失败，请检查网络</Text>
          <View
            className={styles.retryBtn}
            style={{ marginTop: 24 }}
            onClick={() => {
              setLoading(true);
              setRetryTick(t => t + 1);
            }}
          >
            <Text>重试</Text>
          </View>
        </View>
      ) : chapterVerses.length > 0 ? (
        chapterVerses.map(verse => (
          <VerseCard
            key={verse.id}
            verse={verse}
            isRead={readVerseIds.includes(verse.id)}
            hasNote={noteVerseIds.includes(verse.id)}
            onClick={handleVerseClick}
          />
        ))
      ) : (
        <Text className={styles.emptyTip}>本篇内容正在整理中...</Text>
      )}
    </View>
  </ScrollView>
);
```

```
);
};
export default ChapterDetailPage;
/* ===== 文件: src/packageContent/pages/verseDetail/index.tsx ===== */
import React, { useState, useMemo, useEffect, useCallback } from 'react';
import { View, Text, ScrollView } from '@tarojs/components';
import Taro, { useRouter, useDidShow, useShareAppMessage } from '@tarojs/taro';
import classNames from 'classnames';
import styles from './index.module.scss';
import Skeleton from '@components/Skeleton';
import { loadVerse, versesIndex } from '@data/versesLoader';
import type { Verse, MyNote } from '@types';
import { chapters } from '@data/chapters';
import { useProgress } from '@hooks/useProgress';
import { getProgress, deleteNote } from '@utils/storage';
import { getSettings, saveSettings, type FontSize } from '@utils/settings';
// 将文本中的换行符（支持 \n 字面量和真实换行）拆分为行数组渲染
function renderLines(text: string) {
  if (!text) return null;
  const lines = text.replace(/\\n/g, '\n').split('\n');
  return lines.map((line, i) => (
    <Text key={i} selectable style={{ display: 'block' }}>
      {line || '\u00A0'}
    </Text>
  ));
}
}
const VerseDetailPage: React.FC = () => {
  const router = useRouter();
  const [verseId, setVerseId] = useState(() => Number(router.params.id || '101'));
  const { isRead, markRead } = useProgress();
  // 分享: 转发当前章句, 标题用原文截取, path 带 verseId 供他人打开
  useShareAppMessage(() => {
    const title = verse
      ? verse.original.slice(0, 24) + (verse.original.length > 24 ? '...' : '')
      : '论语学习';
    return {
      title,
      path: `/packageContent/pages/verseDetail/index?id=${verseId}`
    };
  });
  const [verse, setVerse] = useState<Verse | null>(null);
  const [loading, setLoading] = useState(true);
  const [loadFailed, setLoadFailed] = useState(false);
  const [retryTick, setRetryTick] = useState(0); // 重试计数, 变化时重新触发加载
  const [myNotes, setMyNotes] = useState<MyNote[]>([]);
  const [fontSize, setFontSize] = useState<FontSize>(() => getSettings().fontSize);
  // 刷新笔记列表
  const refreshNotes = useCallback(() => {
    const progress = getProgress();
    setMyNotes(progress.myNotes.filter(n => n.verseId === verseId));
  }, [verseId]);
```

{ id: 299, chapterId: 8, order: 11, original: "子曰：‘如有周公之才之美，使骄且吝，其余不足观也已’" },

{ id: 300, chapterId: 8, order: 12, original: "子曰：‘三年学，不至于谷，不易得也。’" },

{ id: 301, chapterId: 8, order: 13, original: "子曰：‘笃信好学，守死善道。危邦不入，乱邦不居。天’" },

{ id: 302, chapterId: 8, order: 14, original: "子曰：‘不在其位，不谋其政。’" },

{ id: 303, chapterId: 8, order: 15, original: "子曰：‘师挚之始，《关雎》之乱，洋洋乎盈耳哉！’" },

{ id: 304, chapterId: 8, order: 16, original: "子曰：‘狂而不直，侗而不愿，怍怍而不信，吾不知之矣’" },

{ id: 305, chapterId: 8, order: 17, original: "子曰：‘学如不及，犹恐失之。’" },

{ id: 306, chapterId: 8, order: 18, original: "子曰：‘巍巍乎！舜、禹之有天下也而不与焉。’" },

{ id: 307, chapterId: 8, order: 19, original: "子曰：‘大哉尧之为君也！巍巍乎，唯天为大，唯尧则之’" },

{ id: 308, chapterId: 8, order: 20, original: "舜有臣五人而天下治。武王曰：‘予有乱臣十人。’孔子" },

{ id: 309, chapterId: 8, order: 21, original: "子曰：‘禹，吾无间然矣。菲饮食，而致孝乎鬼神；恶衣" },

{ id: 310, chapterId: 9, order: 1, original: "子罕言利与命与仁。" },

{ id: 311, chapterId: 9, order: 2, original: "达巷党人曰：‘大哉孔子！博学而无所成名。’子闻之，谓" },

{ id: 312, chapterId: 9, order: 3, original: "子曰：‘麻冕，礼也；今也纯，俭，吾从众。拜下，礼也；" },

{ id: 313, chapterId: 9, order: 4, original: "子绝四：毋意、毋必、毋固、毋我。" },

{ id: 314, chapterId: 9, order: 5, original: "子畏于匡，曰：‘文王既没，文不在兹乎？天之将丧斯文也" },

{ id: 315, chapterId: 9, order: 6, original: "太宰问于子贡曰：‘夫子圣者与，何其多能也？’子贡曰：" },

{ id: 316, chapterId: 9, order: 7, original: "牢曰：‘子云：‘吾不试，故艺。’" },

{ id: 317, chapterId: 9, order: 8, original: "子曰：‘吾有知乎哉？无知也。有鄙夫问于我，空空如也。" },

{ id: 318, chapterId: 9, order: 9, original: "子曰：‘凤鸟不至，河不出图，吾已矣夫！’" },

{ id: 319, chapterId: 9, order: 10, original: "子见齐衰者、冕衣裳者与瞽者，见之，虽少，必作，过之" },

{ id: 320, chapterId: 9, order: 11, original: "颜渊喟然叹曰：‘仰之弥高，钻之弥坚。瞻之在前，忽焉" },

{ id: 321, chapterId: 9, order: 12, original: "子疾病，子路使门人为臣。病间，曰：‘久矣哉，由之行" },

{ id: 322, chapterId: 9, order: 13, original: "子贡曰：‘有美玉于斯，韞椟而藏诸？求善贾而沽诸？’" },

{ id: 323, chapterId: 9, order: 14, original: "子欲居九夷。或曰：‘陋，如之何？’子曰：‘君子居之" },

{ id: 324, chapterId: 9, order: 15, original: "子曰：‘吾自卫反鲁，然后乐正，《雅》、《颂》各得其" },

{ id: 325, chapterId: 9, order: 16, original: "子曰：‘出则事公卿，入则事父兄，丧事不敢不勉，不为" },

{ id: 326, chapterId: 9, order: 17, original: "子在川上曰：‘逝者如斯夫！不舍昼夜。’" },

{ id: 327, chapterId: 9, order: 18, original: "子曰：‘吾未见好德如好色者也。’" },

{ id: 328, chapterId: 9, order: 19, original: "子曰：‘譬如为山，未成一簣，止，吾止也；譬如平地，" },

{ id: 329, chapterId: 9, order: 20, original: "子曰：‘语之而不惰者，其回也与！’" },

{ id: 330, chapterId: 9, order: 21, original: "子谓颜渊，曰：‘惜乎！吾见其进也，未见其止也。’" },

{ id: 331, chapterId: 9, order: 22, original: "子曰：‘苗而不秀者有矣夫，秀而不实者有矣夫。’" },

{ id: 332, chapterId: 9, order: 23, original: "子曰：‘后生可畏，焉知来者之不如今也？四十、五十而" },

{ id: 333, chapterId: 9, order: 24, original: "子曰：‘法语之言，能无从乎？改之为贵。巽与之言，能" },

{ id: 334, chapterId: 9, order: 25, original: "子曰：‘主忠信。毋友不如己者，过，则勿惮改。’" },

{ id: 335, chapterId: 9, order: 26, original: "子曰：‘三军可夺帅也，匹夫不可夺志也。’" },

{ id: 336, chapterId: 9, order: 27, original: "子曰：‘衣敝缊袍，与衣狐貉者立而不耻者，其由也与！" },

{ id: 337, chapterId: 9, order: 28, original: "子曰：‘岁寒，然后知松柏之后凋也。’" },

{ id: 338, chapterId: 9, order: 29, original: "子曰：‘知者不惑，仁者不忧，勇者不惧。’" },

{ id: 339, chapterId: 9, order: 30, original: "子曰：‘可与共学，未可与适道；可与适道，未可与立；" },

{ id: 340, chapterId: 9, order: 31, original: "‘唐棣之华，偏其反而。岂不尔思？室是远尔。’子曰：" },

{ id: 341, chapterId: 10, order: 1, original: "孔子于乡党，恂恂如也，似不能言者；其在宗庙朝廷，便" },

{ id: 342, chapterId: 10, order: 2, original: "朝，与下大夫言，侃侃如也；与上大夫言，誾誾如也。君" },

{ id: 343, chapterId: 10, order: 3, original: "君召使摈，色勃如也，足躩如也。揖所与立，左右手，衣" },

{ id: 344, chapterId: 10, order: 4, original: "入公门，鞠躬如也，如不容。立不中门，行不履闕。过位" },

{ id: 345, chapterId: 10, order: 5, original: "执圭，鞠躬如也，如不胜。上如揖，下如授。勃如战色，" },

{ id: 346, chapterId: 10, order: 6, original: "君子不以绀緌饰，红紫不以为褻服。当暑，袗絺綌，必表而" },

{ id: 347, chapterId: 10, order: 7, original: "齐，必有明衣，布。齐必变食，居必迁坐。" },

{ id: 348, chapterId: 10, order: 8, original: "食不厌精，脍不厌细。食饐而餲，鱼馁而肉败，不食；色

{ id: 349, chapterId: 10, order: 9, original: "祭于公，不宿肉。祭肉不出三日，出三日不食之矣。" },
{ id: 350, chapterId: 10, order: 10, original: "食不语，寝不言。" },
{ id: 351, chapterId: 10, order: 11, original: "虽疏食菜羹，瓜祭，必齐如也。" },
{ id: 352, chapterId: 10, order: 12, original: "席不正，不坐。" },
{ id: 353, chapterId: 10, order: 13, original: "乡人饮酒，杖者出，斯出矣。" },
{ id: 354, chapterId: 10, order: 14, original: "乡人傺，朝服而立于阼阶。" },
{ id: 355, chapterId: 10, order: 15, original: "问人于他邦，再拜而送之。" },
{ id: 356, chapterId: 10, order: 16, original: "康子馈药，拜而受之。曰：“丘未达，不敢尝。” " },
{ id: 357, chapterId: 10, order: 17, original: "厩焚，子退朝，曰：“伤人乎？”不问马。" },
{ id: 358, chapterId: 10, order: 18, original: "君赐食，必正席先尝之。君赐腥，必熟而荐之。君赐生，
{ id: 359, chapterId: 10, order: 19, original: "疾，君视之，东首，加朝服，拖绅。" },
{ id: 360, chapterId: 10, order: 20, original: "君命召，不俟驾行矣。" },
{ id: 361, chapterId: 10, order: 21, original: "入太庙，每事问。" },
{ id: 362, chapterId: 10, order: 22, original: "朋友死，无所归，曰：“于我殡。” " },
{ id: 363, chapterId: 10, order: 23, original: "朋友之馈，虽车马，非祭肉，不拜。" },
{ id: 364, chapterId: 10, order: 24, original: "寝不尸，居不容。" },
{ id: 365, chapterId: 10, order: 25, original: "见齐衰者，虽狎，必变。见冕者与瞽者，虽褻，必以貌。
{ id: 366, chapterId: 10, order: 26, original: "升车，必正立，执绥。车中不内顾，不疾言，不亲指。"
{ id: 367, chapterId: 10, order: 27, original: "色斯举矣，翔而后集。曰：“山梁雌雉，时哉时哉！”子
{ id: 368, chapterId: 11, order: 1, original: "子曰：“先进于礼乐，野人也；后进于礼乐，君子也。如
{ id: 369, chapterId: 11, order: 2, original: "子曰：“从我于陈、蔡者，皆不及门也。”德行：颜渊，
{ id: 371, chapterId: 11, order: 3, original: "子曰：“回也非助我者也，于吾言无所不说。” " },
{ id: 372, chapterId: 11, order: 4, original: "子曰：“孝哉闵子骞！人不间于其父母昆弟之言。” " },
{ id: 373, chapterId: 11, order: 5, original: "南容三复白圭，孔子以其兄之子妻之。" },
{ id: 374, chapterId: 11, order: 6, original: "季康子问：“弟子孰为好学？”孔子对曰：“有颜回者好
{ id: 375, chapterId: 11, order: 7, original: "颜渊死，颜路请子之车以为之椁。子曰：“才不才，亦各
{ id: 376, chapterId: 11, order: 8, original: "颜渊死，子曰：“噫！天丧予！天丧予！” " },
{ id: 377, chapterId: 11, order: 9, original: "颜渊死，子哭之恸，从者曰：“子恸矣！”曰：“有恸乎
{ id: 378, chapterId: 11, order: 10, original: "颜渊死，门人欲厚葬之，子曰：“不可。”门人厚葬之，
{ id: 379, chapterId: 11, order: 11, original: "季路问事鬼神，子曰：“未能事人，焉能事鬼？”曰：
{ id: 380, chapterId: 11, order: 12, original: "闵子侍侧，闾闾如也；子路，行行如也；冉有、子贡，侃
{ id: 381, chapterId: 11, order: 13, original: "鲁人为长府，闵子骞曰：“仍旧贯如之何？何必改作？”
{ id: 382, chapterId: 11, order: 14, original: "子曰：“由之瑟，奚为于丘之门？”门人不敬子路，子曰
{ id: 383, chapterId: 11, order: 15, original: "子贡问：“师与商也孰贤？”子曰：“师也过，商也不及
{ id: 384, chapterId: 11, order: 16, original: "季氏富于周公，而求也为之聚敛而附益之。子曰：“非吾
{ id: 385, chapterId: 11, order: 17, original: "柴也愚，参也鲁，师也辟，由也喭。” },
{ id: 386, chapterId: 11, order: 18, original: "子曰：“回也其庶乎，屡空。赐不受命而货殖焉，亿则屡
{ id: 387, chapterId: 11, order: 19, original: "子张问善人之道，子曰：“不践迹，亦不入于室。” " },
{ id: 388, chapterId: 11, order: 20, original: "子曰：“论笃是与，君子者乎，色庄者乎？” " },
{ id: 389, chapterId: 11, order: 21, original: "子路问：“闻斯行诸？”子曰：“有父兄在，如之何其闻
{ id: 390, chapterId: 11, order: 22, original: "子畏于匡，颜渊后。子曰：“吾以女为死矣！”曰：“子
{ id: 391, chapterId: 11, order: 23, original: "季子然问：“仲由、冉求可谓大臣与？”子曰：“吾以子
{ id: 392, chapterId: 11, order: 24, original: "子路使子羔为费宰，子曰：“贼夫人之子。”子路曰：“
{ id: 393, chapterId: 11, order: 25, original: "子路、曾皙、冉有、公西华侍坐，子曰：“以吾一日长乎
{ id: 394, chapterId: 12, order: 1, original: "颜渊问仁，子曰：“克己复礼为仁。一日克己复礼，天下
{ id: 395, chapterId: 12, order: 2, original: "仲弓问仁，子曰：“出门如见大宾，使民如承大祭。己所
{ id: 396, chapterId: 12, order: 3, original: "司马牛问仁，子曰：“仁者，其言也讱。”曰：“其言也
{ id: 397, chapterId: 12, order: 4, original: "司马牛问君子，子曰：“君子不忧不惧。”曰：“不忧不
{ id: 398, chapterId: 12, order: 5, original: "司马牛忧曰：“人皆有兄弟，我独亡。”子夏曰：“商闻
{ id: 399, chapterId: 12, order: 6, original: "子张问明，子曰：“浸润之谮，肤受之愬，不行焉，可谓

{ id: 400, chapterId: 12, order: 7, original: "子贡问政，子曰：“足食，足兵，民信之矣。”子贡曰：

{ id: 401, chapterId: 12, order: 8, original: "棘子成曰：“君子质而已矣，何以文为？”子贡曰：“惜

{ id: 402, chapterId: 12, order: 9, original: "哀公问于有若曰：“年饥，用不足，如之何？”有若对曰

{ id: 403, chapterId: 12, order: 10, original: "子张问崇德、辨惑，子曰：“主忠信，徙义，崇德也。爱

{ id: 404, chapterId: 12, order: 11, original: "齐景公问政于孔子，孔子对曰：“君君，臣臣，父父，子

{ id: 405, chapterId: 12, order: 12, original: "子曰：“片言可以折狱者，其由也与？”子路无宿诺。”

{ id: 406, chapterId: 12, order: 13, original: "子曰：“听讼，吾犹人也。必也使无讼乎。” " }，

{ id: 407, chapterId: 12, order: 14, original: "子张问政，子曰：“居之无倦，行之以忠。” " }，

{ id: 408, chapterId: 12, order: 15, original: "子曰：“博学于文，约之以礼，亦可以弗畔矣夫。” " }，

{ id: 409, chapterId: 12, order: 16, original: "子曰：“君子成人之美，不成人之恶；小人反是。” " }，

{ id: 410, chapterId: 12, order: 17, original: "季康子问政于孔子，孔子对曰：“政者，正也。子帅以正

{ id: 411, chapterId: 12, order: 18, original: "季康子患盗，问于孔子。孔子对曰：“苟子之不欲，虽赏

{ id: 412, chapterId: 12, order: 19, original: "季康子问政于孔子曰：“如杀无道以就有道，何如？”孔

{ id: 413, chapterId: 12, order: 20, original: "子张问：“士何如斯可谓之达矣？”子曰：“何哉尔所谓

{ id: 414, chapterId: 12, order: 21, original: "樊迟从游于舞雩之下，曰：“敢问崇德、修慝、辨惑。”

{ id: 415, chapterId: 12, order: 22, original: "樊迟问仁，子曰：“爱人。”问知，子曰：“知人。”樊

{ id: 416, chapterId: 12, order: 23, original: "子贡问友，子曰：“忠告而善道之，不可则止，毋自辱焉

{ id: 417, chapterId: 12, order: 24, original: "曾子曰：“君子以文会友，以友辅仁。” " }，

{ id: 418, chapterId: 13, order: 1, original: "子路问政，子曰：“先之，劳之。”请益，曰：“无倦。

{ id: 419, chapterId: 13, order: 2, original: "仲弓为季氏宰，问政，子曰：“先有司，赦小过，举贤才

{ id: 420, chapterId: 13, order: 3, original: "子路曰：“卫君待子而为政，子将奚先？”子曰：“必也

{ id: 421, chapterId: 13, order: 4, original: "樊迟请学稼，子曰：“吾不如老农。”请学为圃，曰：“

{ id: 422, chapterId: 13, order: 5, original: "子曰：“诵《诗》三百，授之以政，不达；使于四方，不

{ id: 423, chapterId: 13, order: 6, original: "子曰：“其身正，不令而行；其身不正，虽令不从。” " }

{ id: 424, chapterId: 13, order: 7, original: "子曰：“鲁卫之政，兄弟也。” " }，

{ id: 425, chapterId: 13, order: 8, original: "子谓卫公子荆，“善居室。始有，曰：‘苟合矣。’少有

{ id: 426, chapterId: 13, order: 9, original: "子适卫，冉有仆，子曰：“庶矣哉！”冉有曰：“既庶矣

{ id: 427, chapterId: 13, order: 10, original: "子曰：“苟有用我者，期月而已可也，三年有成。” " }，

{ id: 428, chapterId: 13, order: 11, original: "子曰：“‘善人为邦百年，亦可以胜残去杀矣。’诚哉是

{ id: 429, chapterId: 13, order: 12, original: "子曰：“如有王者，必世而后仁。” " }，

{ id: 430, chapterId: 13, order: 13, original: "子曰：“苟正其身矣，于从政乎何有？不能正其身，如正

{ id: 431, chapterId: 13, order: 14, original: "冉子退朝，子曰：“何晏也？”对曰：“有政。”子曰：

{ id: 432, chapterId: 13, order: 15, original: "定公问：“一言而可以兴邦，有诸？”孔子对曰：“言不

{ id: 433, chapterId: 13, order: 16, original: "叶公问政，子曰：“近者说，远者来。” " }，

{ id: 434, chapterId: 13, order: 17, original: "子夏为莒父宰，问政，子曰：“无欲速，未见小利。欲速

{ id: 435, chapterId: 13, order: 18, original: "叶公语孔子曰：“吾党有直躬者，其父攘羊，而子证之。

{ id: 436, chapterId: 13, order: 19, original: "樊迟问仁，子曰：“居处恭，执事敬，与人忠。虽之夷狄

{ id: 437, chapterId: 13, order: 20, original: "子贡问曰：“何如斯可谓之士矣？”子曰：“行己有耻，

{ id: 438, chapterId: 13, order: 21, original: "子曰：“不得中行而与之，必也狂狷乎！狂者进取，狷者

{ id: 439, chapterId: 13, order: 22, original: "子曰：“南人有言曰：‘人而无恒，不可以作巫医。’善

{ id: 440, chapterId: 13, order: 23, original: "子曰：“君子和而不同，小人同而不和。” " }，

{ id: 441, chapterId: 13, order: 24, original: "子贡问曰：“乡人皆好之，何如？”子曰：“未可也。”

{ id: 442, chapterId: 13, order: 25, original: "子曰：“君子易事而难说也，说之不以道不说也，及其使

{ id: 443, chapterId: 13, order: 26, original: "子曰：“君子泰而不骄，小人骄而不泰。” " }，

{ id: 444, chapterId: 13, order: 27, original: "子曰：“刚、毅、木、讷近仁。” " }，

{ id: 445, chapterId: 13, order: 28, original: "子路问曰：“何如斯可谓之士矣？”子曰：“切切偲偲，

{ id: 446, chapterId: 13, order: 29, original: "子曰：“善人教民七年，亦可以即戎矣。” " }，

{ id: 447, chapterId: 13, order: 30, original: "子曰：“以不教民战，是谓弃之。” " }，

{ id: 448, chapterId: 14, order: 1, original: "宪问耻，子曰：“邦有道，谷；邦无道，谷，耻也。” “

{ id: 449, chapterId: 14, order: 2, original: "子曰：“士而怀居，不足以为士矣。” " }，

{ id: 450, chapterId: 14, order: 3, original: "子曰：‘邦有道，危言危行；邦无道，危行言孙。’" },
{ id: 451, chapterId: 14, order: 4, original: "子曰：‘有德者必有言，有言者不必有德。仁者必有勇，
{ id: 452, chapterId: 14, order: 5, original: "南宫适问于孔子曰：‘羿善射，奭荡舟，俱不得其死然；
{ id: 453, chapterId: 14, order: 6, original: "子曰：‘君子而不仁者有矣夫，未有小人而仁者也。’" },
{ id: 454, chapterId: 14, order: 7, original: "子曰：‘爱之，能勿劳乎？忠焉，能勿诲乎？’" },
{ id: 455, chapterId: 14, order: 8, original: "子曰：‘为命，裨谌草创之，世叔讨论之，行人子羽修饰
{ id: 456, chapterId: 14, order: 9, original: "或问子产，子曰：‘惠人也。’问子西，曰：‘彼哉，彼
{ id: 457, chapterId: 14, order: 10, original: "子曰：‘贫而无怨难，富而无骄易。’" },
{ id: 458, chapterId: 14, order: 11, original: "子曰：‘孟公绰为赵、魏老则优，不可以为滕、薛大夫。
{ id: 459, chapterId: 14, order: 12, original: "子路问成人，子曰：‘若臧武仲之知、公绰之不欲、卞庄
{ id: 460, chapterId: 14, order: 13, original: "子问公叔文子于公明贾曰：‘信乎，夫子不言，不笑，不
{ id: 461, chapterId: 14, order: 14, original: "子曰：‘臧武仲以防求为后于鲁，虽曰不要君，吾不信也
{ id: 462, chapterId: 14, order: 15, original: "子曰：‘晋文公谲而不正，齐桓公正而不谲。’" },
{ id: 463, chapterId: 14, order: 16, original: "子路曰：‘桓公杀公子纠，召忽死之，管仲不死，曰未仁
{ id: 464, chapterId: 14, order: 17, original: "子贡曰：‘管仲非仁者与？桓公杀公子纠，不能死，又相
{ id: 465, chapterId: 14, order: 18, original: "公叔文子之臣大夫僎与文子同升诸公，子闻之，曰：‘可
{ id: 466, chapterId: 14, order: 19, original: "子言卫灵公之无道也，康子曰：‘夫如是，奚而不丧？’
{ id: 467, chapterId: 14, order: 20, original: "子曰：‘其言之不怍，则为之也难。’" },
{ id: 468, chapterId: 14, order: 21, original: "陈成子弑简公，孔子沐浴而朝，告于哀公曰：‘陈恒弑其
{ id: 469, chapterId: 14, order: 22, original: "子路问事君，子曰：‘勿欺也，而犯之。’" },
{ id: 470, chapterId: 14, order: 23, original: "子曰：‘君子上达，小人下达。’" },
{ id: 471, chapterId: 14, order: 24, original: "子曰：‘古之学者为己，今之学者为人。’" },
{ id: 472, chapterId: 14, order: 25, original: "蘧伯玉使人于孔子，孔子与之坐而问焉，曰：‘夫子何为
{ id: 473, chapterId: 14, order: 26, original: "子曰：‘不在其位，不谋其政。’曾子曰：‘君子思不出
{ id: 474, chapterId: 14, order: 27, original: "子曰：‘君子耻其言而过其行。’" },
{ id: 475, chapterId: 14, order: 28, original: "子曰：‘君子道者三，我无能焉：仁者不忧，知者不惑，
{ id: 476, chapterId: 14, order: 29, original: "子贡方人，子曰：‘赐也贤乎哉？夫我则不暇。’" },
{ id: 477, chapterId: 14, order: 30, original: "子曰：‘不患人之不己知，患其不能也。’" },
{ id: 478, chapterId: 14, order: 31, original: "子曰：‘不逆诈，不亿不信，抑亦先觉者，是贤乎！’" },
{ id: 479, chapterId: 14, order: 32, original: "微生亩谓孔子曰：‘丘何为是栖栖者与？无乃为佞乎？’
{ id: 480, chapterId: 14, order: 33, original: "子曰：‘骥不称其力，称其德也。’" },
{ id: 481, chapterId: 14, order: 34, original: "或曰：‘以德报怨，何如？’子曰：‘何以报德？以直报
{ id: 482, chapterId: 14, order: 35, original: "子曰：‘莫我知也夫！’子贡曰：‘何为其莫知子也？’
{ id: 483, chapterId: 14, order: 36, original: "公伯寮愬子路于季孙。子服景伯以告，曰：‘夫子固有惑
{ id: 484, chapterId: 14, order: 37, original: "子曰：‘贤者辟世，其次辟地，其次辟色，其次辟言。’
{ id: 485, chapterId: 14, order: 38, original: "子路宿于石门，晨门曰：‘奚自？’子路曰：‘自孔氏。
{ id: 486, chapterId: 14, order: 39, original: "子击磬于卫，有荷蕢而过孔氏之门者，曰：‘有心哉，击
{ id: 487, chapterId: 14, order: 40, original: "子张曰：‘《书》云，‘高宗谅阴，三年不言。’何谓也
{ id: 488, chapterId: 14, order: 41, original: "子曰：‘上好礼，则民易使也。’" },
{ id: 489, chapterId: 14, order: 42, original: "子路问君子，子曰：‘修己以敬。’曰：‘如斯而已乎？
{ id: 490, chapterId: 14, order: 43, original: "原壤夷俟，子曰：‘幼而不孙弟，长而无述焉，老而不死
{ id: 491, chapterId: 14, order: 44, original: "阙党童子将命，或问之曰：‘益者与？’子曰：‘吾见其
{ id: 492, chapterId: 15, order: 1, original: "卫灵公问陈于孔子，孔子对曰：‘俎豆之事，则尝闻之矣
{ id: 493, chapterId: 15, order: 2, original: "在陈绝粮，从者病莫能兴。子路愠见曰：‘君子亦有穷乎
{ id: 494, chapterId: 15, order: 3, original: "子曰：‘赐也，女以予为多学而识之者与？’对曰：‘然
{ id: 495, chapterId: 15, order: 4, original: "子曰：‘由，知德者鲜矣。’" },
{ id: 496, chapterId: 15, order: 5, original: "子曰：‘无为而治者其舜也与！夫何为哉？恭己正南面而
{ id: 497, chapterId: 15, order: 6, original: "子张问行，子曰：‘言忠信，行笃敬，虽蛮貊之邦，行矣
{ id: 498, chapterId: 15, order: 7, original: "子曰：‘直哉史鱼！邦有道如矢，邦无道如矢。君子哉蘧
{ id: 499, chapterId: 15, order: 8, original: "子曰：‘可与言而与之言，失人；不可与言而与之言，

{ id: 500, chapterId: 15, order: 9, original: "子曰：‘志士仁人无求生以害仁，有杀身以成仁。’" },
{ id: 501, chapterId: 15, order: 10, original: "子贡问为仁，子曰：‘工欲善其事，必先利其器。居是邦’
{ id: 502, chapterId: 15, order: 11, original: "颜渊问为邦，子曰：‘行夏之时，乘殷之辂，服周之冕，
{ id: 503, chapterId: 15, order: 12, original: "子曰：‘人无远虑，必有近忧。’" },
{ id: 504, chapterId: 15, order: 13, original: "子曰：‘已矣乎！吾未见好德如好色者也。’" },
{ id: 505, chapterId: 15, order: 14, original: "子曰：‘臧文仲其窃位者与！知柳下惠之贤而不与立也。
{ id: 506, chapterId: 15, order: 15, original: "子曰：‘躬自厚而薄责于人，则远怨矣。’" },
{ id: 507, chapterId: 15, order: 16, original: "子曰：‘不曰‘如之何、如之何’者，吾未如之何也已矣
{ id: 508, chapterId: 15, order: 17, original: "子曰：‘群居终日，言不及义，好行小慧，难矣哉！’" },
{ id: 509, chapterId: 15, order: 18, original: "子曰：‘君子义以为质，礼以行之，孙以出之，信以成之
{ id: 510, chapterId: 15, order: 19, original: "子曰：‘君子病无能焉，不病人之不已知也。’" },
{ id: 511, chapterId: 15, order: 20, original: "子曰：‘君子疾没世而名不称焉。’" },
{ id: 512, chapterId: 15, order: 21, original: "子曰：‘君子求诸己，小人求诸人。’" },
{ id: 513, chapterId: 15, order: 22, original: "子曰：‘君子矜而不争，群而不党。’" },
{ id: 514, chapterId: 15, order: 23, original: "子曰：‘君子不以言举人，不以人废言。’" },
{ id: 515, chapterId: 15, order: 24, original: "子贡问曰：‘有一言而可以终身行之者乎？’子曰：‘其
{ id: 516, chapterId: 15, order: 25, original: "子曰：‘吾之于人也，谁毁谁誉？如有所誉者，其有所试
{ id: 517, chapterId: 15, order: 26, original: "子曰：‘吾犹及史之阙文也，有马者借人乘之，今亡矣夫
{ id: 518, chapterId: 15, order: 27, original: "子曰：‘巧言乱德，小不忍，则乱大谋。’" },
{ id: 519, chapterId: 15, order: 28, original: "子曰：‘众恶之，必察焉；众好之，必察焉。’" },
{ id: 520, chapterId: 15, order: 29, original: "子曰：‘人能弘道，非道弘人。’" },
{ id: 521, chapterId: 15, order: 30, original: "子曰：‘过而不改，是谓过矣。’" },
{ id: 522, chapterId: 15, order: 31, original: "子曰：‘吾尝终日不食、终夜不寝以思，无益，不如学也
{ id: 523, chapterId: 15, order: 32, original: "子曰：‘君子谋道不谋食。耕也馁在其中矣，学也禄在其
{ id: 524, chapterId: 15, order: 33, original: "子曰：‘知及之，仁不能守之，虽得之，必失之。知及之
{ id: 525, chapterId: 15, order: 34, original: "子曰：‘君子不可小知而可大受也，小人不可大受而可小
{ id: 526, chapterId: 15, order: 35, original: "子曰：‘民之于仁也，甚于水火。水火，吾见蹈而死者矣
{ id: 527, chapterId: 15, order: 36, original: "子曰：‘当仁不让于师。’" },
{ id: 528, chapterId: 15, order: 37, original: "子曰：‘君子贞而不谅。’" },
{ id: 529, chapterId: 15, order: 38, original: "子曰：‘事君，敬其事而后其食。’" },
{ id: 530, chapterId: 15, order: 39, original: "子曰：‘有教无类。’" },
{ id: 531, chapterId: 15, order: 40, original: "子曰：‘道不同，不相为谋。’" },
{ id: 532, chapterId: 15, order: 41, original: "子曰：‘辞达而已矣。’" },
{ id: 533, chapterId: 15, order: 42, original: "师冕见，及阶，子曰：‘阶也。’及席，子曰：‘席也。
{ id: 534, chapterId: 16, order: 1, original: "季氏将伐颛臾，冉有、季路见于孔子，曰：‘季氏将有事
{ id: 535, chapterId: 16, order: 2, original: "孔子曰：‘天下有道，则礼乐征伐自天子出；天下无道，
{ id: 536, chapterId: 16, order: 3, original: "孔子曰：‘禄之去公室五世矣，政逮于大夫四世矣，故夫
{ id: 537, chapterId: 16, order: 4, original: "孔子曰：‘益者三友，损者三友。友直、友谅、友多闻，
{ id: 538, chapterId: 16, order: 5, original: "孔子曰：‘益者三乐，损者三乐。乐节礼乐、乐道人之善
{ id: 539, chapterId: 16, order: 6, original: "孔子曰：‘侍于君子有三愆：言未及之而言谓之躁，言及
{ id: 540, chapterId: 16, order: 7, original: "孔子曰：‘君子有三戒：少之时，血气未定，戒之在色；
{ id: 541, chapterId: 16, order: 8, original: "孔子曰：‘君子有三畏：畏天命，畏大人，畏圣人之言。
{ id: 542, chapterId: 16, order: 9, original: "孔子曰：‘生而知之者上也，学而知之者次也；困而学之
{ id: 543, chapterId: 16, order: 10, original: "孔子曰：‘君子有九思：视思明，听思聪，色思温，貌思
{ id: 544, chapterId: 16, order: 11, original: "孔子曰：‘见善如不及，见不善如探汤；吾见其人矣。吾
{ id: 545, chapterId: 16, order: 12, original: "齐景公有马千驷，死之日，民无德而称焉；伯夷、叔齐饿
{ id: 546, chapterId: 16, order: 13, original: "陈亢问于伯鱼曰：‘子亦有异闻乎？’对曰：‘未也。尝
{ id: 547, chapterId: 16, order: 14, original: "邦君之妻，君称之曰夫人，夫人自称曰小童；邦人称之曰
{ id: 548, chapterId: 17, order: 1, original: "阳货欲见孔子，孔子不见，归孔子豚。孔子时其亡也而往
{ id: 549, chapterId: 17, order: 2, original: "子曰：‘性相近也，习相远也。’" },

{ id: 550, chapterId: 17, order: 3, original: "子曰：“唯上知与下愚不移。” " },

{ id: 551, chapterId: 17, order: 4, original: "子之武城，闻弦歌之声。夫子莞尔而笑，曰：“割鸡焉用

{ id: 552, chapterId: 17, order: 5, original: "公山弗扰以费畔，召，子欲往。子路不说，曰：“末之也

{ id: 553, chapterId: 17, order: 6, original: "子张问仁于孔子，孔子曰：“能行五者于天下为仁矣。”

{ id: 554, chapterId: 17, order: 7, original: "佛肸召，子欲往。子路曰：“昔者由也闻诸夫子曰。亲于

{ id: 555, chapterId: 17, order: 8, original: "子曰：“由也，女闻六言六蔽矣乎？”对曰：“未也。”

{ id: 556, chapterId: 17, order: 9, original: "子曰：“小子何莫学夫诗！诗，可以兴，可以观，可以群

{ id: 557, chapterId: 17, order: 10, original: "子谓伯鱼曰：“女为《周南》、《召南》矣乎？人而不为

{ id: 558, chapterId: 17, order: 11, original: "子曰：“礼云礼云，玉帛云乎哉？乐云乐云，钟鼓云乎哉

{ id: 559, chapterId: 17, order: 12, original: "子曰：“色厉而内荏，譬诸小人，其犹穿窬之盗也与？”

{ id: 560, chapterId: 17, order: 13, original: "子曰：“乡愿，德之贼也。” " },

{ id: 561, chapterId: 17, order: 14, original: "子曰：“道听而涂说，德之弃也。” " },

{ id: 562, chapterId: 17, order: 15, original: "子曰：“鄙夫可与事君也与哉？其未得之也，患得之；既

{ id: 563, chapterId: 17, order: 16, original: "子曰：“古者民有三疾，今也或是之亡也。古之狂也肆，

{ id: 564, chapterId: 17, order: 17, original: "子曰：“巧言令色，鲜矣仁。” " },

{ id: 565, chapterId: 17, order: 18, original: "子曰：“恶紫之夺朱也，恶郑声之乱雅乐也，恶利口之覆

{ id: 566, chapterId: 17, order: 19, original: "子曰：“子欲无言。”子贡曰：“子如不言，则小子何述

{ id: 567, chapterId: 17, order: 20, original: "孺悲欲见孔子，孔子辞以疾。将命者出户，取瑟而歌，使

{ id: 568, chapterId: 17, order: 21, original: "宰我问：“三年之丧，期已久矣！君子三年不为礼，礼必

{ id: 569, chapterId: 17, order: 22, original: "子曰：“饱食终日，无所用心，难矣哉！不有博弈者乎？

{ id: 570, chapterId: 17, order: 23, original: "子路曰：“君子尚勇乎？”子曰：“君子义以为上。君子

{ id: 571, chapterId: 17, order: 24, original: "子贡曰：“君子亦有恶乎？”子曰：“有恶。恶称人之恶

{ id: 572, chapterId: 17, order: 25, original: "子曰：“唯女子与小人为难养也，近之则不孙，远之则怨

{ id: 573, chapterId: 17, order: 26, original: "子曰：“年四十而见恶焉，其终也已。” " },

{ id: 574, chapterId: 18, order: 1, original: "微子去之，箕子为之奴，比干谏而死。孔子曰：“殷有三

{ id: 575, chapterId: 18, order: 2, original: "柳下惠为士师，三黜。人曰：“子未可以去乎？”曰：“

{ id: 576, chapterId: 18, order: 3, original: "齐景公待孔子曰：“若季氏，则吾不能。”以季、孟之间

{ id: 577, chapterId: 18, order: 4, original: "齐人归女乐，季桓子受之，三日不朝，孔子行。” },

{ id: 578, chapterId: 18, order: 5, original: "楚狂接舆歌而过孔子曰：“凤兮凤兮，何德之衰！往者不

{ id: 579, chapterId: 18, order: 6, original: "长沮、桀溺耦而耕，孔子过之，使子路问津焉。长沮曰：

{ id: 580, chapterId: 18, order: 7, original: "子路从而后，遇丈人，以杖荷蓑。子路问曰：“子见夫子

{ id: 581, chapterId: 18, order: 8, original: "逸民：伯夷、叔齐、虞仲、夷逸、朱张、柳下惠、少连。

{ id: 582, chapterId: 18, order: 9, original: "太师挚适齐，亚饭干适楚，三饭缭适蔡，四饭缺适秦，鼓

{ id: 583, chapterId: 18, order: 10, original: "周公谓鲁公曰：“君子不施其亲，不使大臣怨乎不以，故

{ id: 584, chapterId: 18, order: 11, original: "周有八士：伯达、伯适、仲突、仲忽、叔夜、叔夏、季随

{ id: 585, chapterId: 19, order: 1, original: "子张曰：“士见危致命，见得思义，祭思敬，丧思哀，其

{ id: 586, chapterId: 19, order: 2, original: "子张曰：“执德不弘，信道不笃，焉能为有？焉能为亡？

{ id: 587, chapterId: 19, order: 3, original: "子夏之门人问交于子张，子张曰：“子夏云何？”对曰：

{ id: 588, chapterId: 19, order: 4, original: "子夏曰：“虽小道必有可观者焉，致远恐泥，是以君子不

{ id: 589, chapterId: 19, order: 5, original: "子夏曰：“日知其所亡，月无忘其所能，可谓好学也已矣

{ id: 590, chapterId: 19, order: 6, original: "子夏曰：“博学而笃志，切问而近思，仁在其中矣。” " }

{ id: 591, chapterId: 19, order: 7, original: "子夏曰：“百工居肆以成其事，君子学以致其道。” " },

{ id: 592, chapterId: 19, order: 8, original: "子夏曰：“小人之过也必文。” " },

{ id: 593, chapterId: 19, order: 9, original: "子夏曰：“君子有三变：望之俨然，即之也温，听其言也

{ id: 594, chapterId: 19, order: 10, original: "子夏曰：“君子信而后劳其民，未信，则以为厉己也；信

{ id: 595, chapterId: 19, order: 11, original: "子夏曰：“大德不逾闲，小德出入可也。” " },

{ id: 596, chapterId: 19, order: 12, original: "子游曰：“子夏之门人小子，当洒扫应对进退则可矣。抑

{ id: 597, chapterId: 19, order: 13, original: "子夏曰：“仕而优则学，学而优则仕。” " },

{ id: 598, chapterId: 19, order: 14, original: "子游曰：“丧致乎哀而止。” " },

{ id: 599, chapterId: 19, order: 15, original: "子游曰：“吾友张也为难能也，然而未仁。” " },

```
{ id: 600, chapterId: 19, order: 16, original: "曾子曰：‘堂堂乎张也，难与并为仁矣。’" },
{ id: 601, chapterId: 19, order: 17, original: "曾子曰：‘吾闻诸夫子，人未有自致者也，必也亲丧乎！’" },
{ id: 602, chapterId: 19, order: 18, original: "曾子曰：‘吾闻诸夫子，孟庄子之孝也，其他可能也；其
{ id: 603, chapterId: 19, order: 19, original: "孟氏使阳肤为士师，问于曾子。曾子曰：‘上失其道，民
{ id: 604, chapterId: 19, order: 20, original: "子贡曰：‘纣之不善，不如是之甚也。是以君子恶居下流
{ id: 605, chapterId: 19, order: 21, original: "子贡曰：‘君子之过也，如日月之食焉。过也人皆见之，
{ id: 606, chapterId: 19, order: 22, original: "卫公孙朝问于子贡曰：‘仲尼焉学？’" 子贡曰：‘文武之
{ id: 607, chapterId: 19, order: 23, original: "叔孙武叔语大夫于朝曰：‘子贡贤于仲尼。’" 子服景伯以
{ id: 608, chapterId: 19, order: 24, original: "叔孙武叔毁仲尼，子贡曰：‘无以为也，仲尼不可毁也。
{ id: 609, chapterId: 19, order: 25, original: "陈子禽谓子贡曰：‘子为恭也，仲尼岂贤于子乎？’" 子贡
{ id: 610, chapterId: 20, order: 1, original: "尧曰：‘咨！尔舜！天之历数在尔躬，允执其中。四海困
{ id: 611, chapterId: 20, order: 2, original: "子张问于孔子曰：‘何如斯可以从政矣？’" 子曰：‘尊五
{ id: 612, chapterId: 20, order: 3, original: "孔子曰：‘不知命，无以为君子也；不知礼，无以立也；
];
/* ===== 文件：src/data/versesLoader.ts ===== */
import type { Verse } from '@types';
import { versesIndex, type VerseIndex } from './versesIndex';
import { VERSES_BLOB_B64 } from './versesData.compressed';
import { base64ToBytes, inflateRaw, utf8Decode } from '@utils/inflate';
// 全量章句数据：懒解压（首次调用时解码一次，之后命中缓存）
// 背景：20 个篇章 TS 编译后 ~2.9MB 撑爆功能模块上限，改为构建时 deflate 压缩（~0.8MB），
// 运行时用 src/utils/inflate.ts 解压（纯 JS 极简实现，首解压仅一次）
let fullData: Verse[] | null = null;
// 单条查询索引（id → verse）
let verseMap: Map<number, Verse> | null = null;
// 篇章缓存（保持 loadChapter 的"同一篇章二次加载返回同一引用"语义）
const chapterCache = new Map<number, Verse[]>();
/** 解压并解析全量章句数据（首次调用执行） */
function getFullData(): Verse[] {
  if (!fullData) {
    const bytes = base64ToBytes(VERSES_BLOB_B64);
    const json = utf8Decode(inflateRaw(bytes));
    const data = JSON.parse(json) as { verses: Verse[] };
    fullData = data.verses;
  }
  return fullData;
}
// 异步加载某篇完整数据（保持异步签名，调用方无需改动）
export async function loadChapter(chapterId: number): Promise<Verse[]> {
  if (chapterCache.has(chapterId)) {
    return chapterCache.get(chapterId)!;
  }
  const verses = getFullData().filter(v => v.chapterId === chapterId);
  chapterCache.set(chapterId, verses);
  return verses;
}
// 异步加载单条经文
export async function loadVerse(verseId: number): Promise<Verse | null> {
  if (!verseMap) {
    const map = new Map<number, Verse>();
```

```
for (const v of getFullData()) {
  map.set(v.id, v);
}
verseMap = map;
}
return verseMap.get(verseId) || null;
}
// 异步加载全部篇章完整数据（用于深度搜索 commentary），利用缓存
export async function loadAllVerses(): Promise<Verse[]> {
  return getFullData();
}
// 轻量索引（同步，用于列表和搜索）
export { versesIndex };
export type { VerseIndex };
/* ===== 文件: src/data/dailyRecommend.ts ===== */
import { DailyRecommend, Chapter } from '@types';
import { chapters } from '@data/chapters';
import { versesIndex, type VerseIndex } from '@data/versesIndex';
// 主题内章句列表缓存（每篇句数固定，避免换一批高频 filter 重复扫描 0(509)）
const chapterVersesCache = new Map<number, VerseIndex[]>();
function getChapterVerses(chapterId: number): VerseIndex[] {
  let verses = chapterVersesCache.get(chapterId);
  if (!verses) {
    verses = versesIndex.filter(v => v.chapterId === chapterId);
    chapterVersesCache.set(chapterId, verses);
  }
  return verses;
}
// 计算一年中的第几天（0-365，按本地时区；1月1日为第0天）。
// 修复 off-by-one: 原实现 new Date(y, 0, 0) 使第0天落在去年12-31，导致 dailyRecommends[0] 全年不被
function getDayOfYear(date: Date): number {
  const start = new Date(date.getFullYear(), 0, 1);
  const diff = date.getTime() - start.getTime();
  return Math.floor(diff / (1000 * 60 * 60 * 24));
}
// 方案 E: 按主题（篇章）轮换——20 篇 20 天一轮，每天主题不同，用户顺着二十篇走
function getTodayChapter(date: Date): Chapter {
  const themeIndex = getDayOfYear(date) % chapters.length;
  return chapters[themeIndex];
}
// 主题内确定性选句：同一天所有用户看到同一句（可分享、可运营）；
// 种子含年份，跨年同月同日句子不同；相邻天主题不同天然不重复
function pickVerseByDate(chapterId: number, date: Date): VerseIndex {
  const verses = getChapterVerses(chapterId);
  const seed = date.getFullYear() * 1000 + getDayOfYear(date);
  let h = seed;
  h = (h ^ (h >>> 13)) * 0x5bd1e995; // 简单整数混淆，保证相邻天结果跳变
  h = (h ^ (h >>> 15)) >>> 0;
  return verses[h % verses.length];
}
```

```
function toDailyRecommend(verse: VerseIndex, chapter: Chapter): DailyRecommend {
  return {
    verseId: verse.id,
    // 推荐理由: keyPoint 在功能模块完整数据里, 主模块首页拿不到(会破坏功能模块体积设计),
    // 首页暂不展示该字段, 用轻量索引可得的篇内位置文案填充, 预留给未来
    reason: `第 ${verse.order} 句 · 本篇共 ${chapter.verseCount} 句`,
    theme: chapter.theme,
    chapterTitle: chapter.title,
    chapterId: chapter.id
  };
}

// 获取今日推荐 (date 可选参数仅供测试注入固定日期, 默认今天)
export function getTodayRecommend(date?: Date): DailyRecommend {
  const today = date || new Date();
  const chapter = getTodayChapter(today);
  return toDailyRecommend(pickVerseByDate(chapter.id, today), chapter);
}

// 换一批: 保持今日主题不变, 同篇章内换一句 (用户主动行为, 允许随机)
export function getAlternativeRecommend(chapterId: number, excludeVerseId?: number): DailyRecommend {
  const all = getChapterVerses(chapterId);
  const pool = all.filter(v => v.id !== excludeVerseId);
  const candidates = pool.length > 0 ? pool : all; // 篇内仅 1 句时回退全池
  const verse = candidates[Math.floor(Math.random() * candidates.length)];
  const chapter = chapters.find(c => c.id === chapterId);
  // 非法 chapterId 兜底 (防御): 回退今日推荐
  if (!verse || !chapter) return getTodayRecommend();
  return toDailyRecommend(verse, chapter);
}

/* ===== 文件: src/config/cloud.ts ===== */
/**
 * 云开发环境配置
 *
 * 开源仓库默认使用占位符; 本地/线上环境请通过「本地覆盖文件」注入真实环境 ID:
 * 1. 复制本文件为 src/config/cloud.local.ts (已被 .gitignore 忽略, 不会提交)
 * 2. 在其中填写你的环境 ID: export const CLOUD_ENV = '你的环境ID';
 *
 * 获取方式: 开发工具 → 云开发 → 设置 → 环境ID
 * 留空时使用默认环境 (仅适用于只有一个环境的开发期, 生产环境强烈建议显式指定)
 */
let localEnv: string | undefined;
try {
  // eslint-disable-next-line @typescript-eslint/no-var-requires
  localEnv = require('@config/cloud.local').CLOUD_ENV;
} catch (e) {
  /* 无本地覆盖文件时使用占位符/默认环境 */
}

export const CLOUD_ENV = localEnv || '';

/* ===== 文件: src/types/index.ts ===== */
// 篇章
export interface Chapter {
```

```
id: number;
title: string;      // 学而第一
subTitle: string;   // 学而
verseCount: number; // 句子数量
description: string; // 篇章简介
theme: string;      // 主题关键词
}
// 句子（一章中的每一条）
export interface Verse {
  id: number;
  chapterId: number;
  order: number;      // 篇章内序号
  original: string;    // 原文
  translation: string; // 译文
  commentary: string; // 注释解读
  keyPoint: string;    // 核心要点
}
// 每日推荐（方案 E：按主题轮换，theme 由篇章唯一映射）
export interface DailyRecommend {
  verselId: number;
  reason: string;      // 推荐理由（篇内位置文案；keyPoint 在功能模块完整数据里，主模块首页暂不展
  theme: string;        // 今日主题（如「学习修身」）
  chapterTitle: string; // 篇章标题（如「学而第一」）
  chapterId: number;    // 篇章 id（「换一批」按同篇换句需要）
}
// 学习进度
export interface LearningProgress {
  readVerselIds: number[]; // 已读句子 ID
  myNotes: MyNote[];       // 我的笔记
  totalReadDays: number;    // 连续学习天数
  lastReadDate?: string;    // 最后学习日期（YYYY-MM-DD）
  lastReadVerselId?: number; // 最后阅读的章句 ID（「接着读」续读入口用）
  deletedNoteIds?: number[]; // 已删除笔记 ID（删除标记，跨端同步时防止旧数据复活）
  likedNoteIds?: string[];   // 存量兼容：历史已点赞的公开心得 ID（去 UGC 后不再新增）
  unlikedNoteIds?: string[]; // 存量兼容：历史已取消点赞的公开心得 ID（去 UGC 后不再新增）
}
// 我的笔记
export interface MyNote {
  id: number;
  verselId: number;
  content: string;
  createTime: string;
  updateTime?: string; // 最后修改时间（YYYY-MM-DD HH:mm, 分钟级），跨设备合并时按此判断最新版本
  tags?: string[];     // 笔记标签
  isPublic?: boolean;   // 存量兼容：历史「公开到社区」标记（去 UGC 后不再写入）
  cloudNoteId?: string; // 存量兼容：历史云端笔记 _id（去 UGC 后不再写入）
}
// 用户信息
export interface UserInfo {
  openId: string;      // 用户 openId
```



```
    nickName: string;          // 昵称
    avatarUrl: string;         // 头像 URL
    gender?: number;           // 性别 0-未知 1-男 2-女
    loginTime: string;         // 最后登录时间
}
// 云端同步结果
export interface SyncResult {
    success: boolean;
    message: string;
    data?: LearningProgress;
}
/* ===== 文件: src/styles/theme.scss ===== */
// =====
// 主题配色变量 —— 书香典雅（论语学习）
// =====
// 品牌主题色：朱砂赭——传统中国色
$color-primary: #b8612d;
$color-primary-light: #d4844a;
$color-primary-dark: #944820;
// 功能色
$color-success: #5b8c5a;
$color-warning: #d4844a;
$color-error: #c0453a;
$color-info: #b8612d;
// 背景色
$color-bg-page: #faf6f0;
$color-bg-card: #ffffff;
$color-bg-hover: #f5ede3;
// 文本色
$color-text-primary: #2c2417;
$color-text-secondary: #6b5d4f;
$color-text-tertiary: #a39685;
$color-text-disabled: #c9c1b5;
$color-text-white: #ffffff;
// 边框色
$color-border: #ede4d8;
$color-border-light: #f0e8db;
$color-divider: #f0e8db;
// 业务相关变量
$color-tag-bg: #f5ede3;
$color-tag-text: #b8612d;
$color-accent: #8c6a4a;
/* ===== 文件: src/styles/variables.scss ===== */
// 必须转发和使用 theme.scss
@forward './theme.scss';
@use './theme.scss' as *;
// =====
// 全局 SCSS 变量（建议直接复用，如有特殊设计需求可按需修改）
// =====
// 屏幕适配（设计稿基准 750rpx）
```

```

$screen-width: 750rpx;
$page-padding: 32rpx; // 页面左右安全边距
$page-spacing: 32rpx;
$max-content-width: 686rpx; // 750 - 32*2, 内容最大宽度
// 间距（8px 网格系统）
$spacing-xs: 8rpx;
$spacing-sm: 16rpx;
$spacing-md: 24rpx;
$spacing-lg: 32rpx;
$spacing-xl: 48rpx;
// 圆角（语义化）
$radius-xs: 4rpx; // 小元素：标签、徽章
$radius-sm: 8rpx; // 卡片内小元素
$radius-md: 12rpx; // 常规卡片
$radius-lg: 16rpx; // 大卡片
$radius-xl: 20rpx; // 特大卡片
$radius-button: 48rpx; // 按钮专用（全圆角，符合主流应用）
$radius-round: 999rpx; // 完全圆形
// 按钮高度（新增）
$button-height-sm: 64rpx; // 小按钮（次要操作）
$button-height-md: 80rpx; // 常规按钮（主流标准）
$button-height-lg: 96rpx; // 大按钮（主要操作、底部固定）
// 阴影（语义化）
$shadow-card: 0 2rpx 12rpx rgba(0, 0, 0, 0.08); // 卡片阴影
$shadow-card-hover: 0 4rpx 20rpx rgba(0, 0, 0, 0.12); // 卡片悬浮
$shadow-float: 0 4rpx 16rpx rgba(0, 0, 0, 0.1); // 浮层阴影（如筛选栏、底部操作栏）
$shadow-popup: 0 8rpx 32rpx rgba(0, 0, 0, 0.15); // 弹窗阴影
// 字体大小
$font-size-xs: 22rpx;
$font-size-sm: 24rpx;
$font-size-md: 28rpx;
$font-size-lg: 32rpx;
$font-size-xl: 36rpx;
$font-size-xxl: 40rpx;
// 行高
$line-height-tight: 1.3;
$line-height-normal: 1.5;
$line-height-loose: 1.8;
// 字重
$font-weight-normal: 400;
$font-weight-medium: 500;
$font-weight-semibold: 600;
$font-weight-bold: 700;
// 过渡动画
$transition-fast: 0.15s ease;
$transition-base: 0.25s ease;
$transition-slow: 0.35s ease;
// =====
// 常用 Mixin
// =====

```

```
// 单行文本省略
@mixin text-ellipsis {
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

// 多行文本省略
@mixin text-ellipsis-multi($lines: 2) {
  display: -webkit-box;
  -webkit-box-orient: vertical;
  -webkit-line-clamp: $lines;
  overflow: hidden;
  text-overflow: ellipsis;
}

// 清除浮动
@mixin clearfix {
  &::after {
    content: '';
    display: table;
    clear: both;
  }
}

// 水平垂直居中
@mixin flex-center {
  display: flex;
  align-items: center;
  justify-content: center;
}

// 横向滚动容器（移动应用兼容）
@mixin scroll-x-container {
  white-space: nowrap;
}

// 横向滚动子项（移动应用兼容，必须配合 scroll-x-container 使用）
@mixin scroll-x-item($width) {
  display: inline-block;
  width: $width;
  vertical-align: top;
  white-space: normal; // 恢复子项内部换行
}

// Button 样式重置与保护
// 使用方法: @include button-reset;
@mixin button-reset {
  padding: 0;
  margin: 0;
  line-height: normal;
  text-align: inherit;
  white-space: nowrap; // 强制不换行
  display: flex;
  align-items: center;
  justify-content: center;
}
```

```
box-sizing: border-box; // 关键：确保 padding 和 border 不会撑大尺寸
&::after {
  border: none;
}
}

/* ===== 文件：src/pages/home/index.module.scss ===== */
@use '@styles/variables.scss' as *;

.container {
  min-height: 100vh;
  padding: 0 $page-padding $spacing-xl;
  background: linear-gradient(180deg, #f5ede3 0%, $color-bg-page 40%);
}

// 顶部标题区
.header {
  padding: $spacing-lg 0 $spacing-md;
}

.appTitle {
  display: block;
  font-size: $font-size-xxl;
  font-weight: $font-weight-semibold;
  color: $color-text-primary;
  margin-bottom: $spacing-xs;
}

.appSubtitle {
  display: block;
  font-size: $font-size-sm;
  color: $color-text-secondary;
}

// 今日推荐卡片
.dailyCard {
  background: linear-gradient(135deg, $color-primary 0%, $color-primary-dark 100%);
  border-radius: $radius-xl;
  padding: $spacing-lg;
  margin-bottom: $spacing-lg;
  box-shadow: 0 8px 24px rgba(184, 97, 45, 0.25);
  position: relative;
  overflow: hidden;
  &::before {
    content: '';
    position: absolute;
    top: -40px;
    right: -40px;
    width: 200px;
    height: 200px;
    border-radius: 50%;
    background: rgba(255, 255, 255, 0.08);
  }
}

// 继续学习卡片（次级样式，浅色底与今日推荐区分）
.continueCard {
```

```
border-radius: $radius-xl;
padding: $spacing-lg;
margin-bottom: $spacing-lg;
background: $color-bg-card;
border: 2px solid $color-divider;
position: relative;
overflow: hidden;
transition: all $transition-base;
&:active {
  opacity: 0.85;
  transform: scale(0.99);
}
.dailyLabelIcon {
  color: $color-primary;
}
.dailyLabelText {
  color: $color-text-secondary;
}
.dailyOriginal {
  color: $color-text-primary;
}
.dailyBtn {
  background: $color-primary;
}
}
.dailyLabel {
  display: flex;
  align-items: center;
  margin-bottom: $spacing-sm;
}
.dailyLabelIcon {
  font-size: $font-size-md;
  margin-right: $spacing-xs;
}
.dailyLabelText {
  font-size: $font-size-sm;
  color: rgba(255, 255, 255, 0.85);
  font-weight: $font-weight-medium;
}
// 主题标签（方案 E）：半透明白底圆角，靠右展示今日主题
.dailyTag {
  margin-left: auto;
  padding: 4px $spacing-sm;
  font-size: $font-size-xs;
  color: $color-primary;
  background: rgba(255, 255, 255, 0.9);
  border-radius: $radius-round;
  font-weight: $font-weight-medium;
  position: relative;
  z-index: 1;
```

```
}
// 篇章标题（方案 E）：主题内确定性选句，展示所属篇章
.dailyChapter {
  display: block;
  font-size: $font-size-xs;
  color: rgba(255, 255, 255, 0.85);
  margin-bottom: $spacing-xs;
  position: relative;
  z-index: 1;
}
.dailyOriginal {
  display: block;
  font-size: $font-size-lg;
  color: $color-text-white;
  line-height: $line-height-loose;
  font-weight: $font-weight-medium;
  margin-bottom: $spacing-sm;
  position: relative;
  z-index: 1;
}
.dailyBtn {
  display: inline-flex;
  align-items: center;
  background: rgba(255, 255, 255, 0.2);
  padding: 12rpx $spacing-lg;
  border-radius: $radius-button;
  position: relative;
  z-index: 1;
  transition: all $transition-base;
  &:active {
    opacity: 0.8;
    transform: scale(0.98);
  }
}
.dailyActions {
  display: flex;
  gap: $spacing-sm;
  position: relative;
  z-index: 1;
}
.dailyBtnText {
  font-size: $font-size-sm;
  color: $color-text-white;
  white-space: nowrap;
}
// 进度概览
.progressSection {
  background: $color-bg-card;
  border-radius: $radius-lg;
  padding: $spacing-lg;
```

```
margin-bottom: $spacing-lg;
box-shadow: $shadow-card;
}
.progressTitle {
  display: block;
  font-size: $font-size-md;
  font-weight: $font-weight-semibold;
  color: $color-text-primary;
  margin-bottom: $spacing-md;
}
.statsRow {
  display: flex;
  justify-content: space-between;
  margin-bottom: $spacing-lg;
}
.statItem {
  display: flex;
  flex-direction: column;
  align-items: center;
  flex: 1;
}
.statNumber {
  font-size: $font-size-xl;
  font-weight: $font-weight-semibold;
  color: $color-primary;
}
.statLabel {
  font-size: $font-size-xs;
  color: $color-text-tertiary;
  margin-top: 4rpx;
}
// 快捷入口
.sectionTitle {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: $spacing-md;
}
.sectionTitleText {
  font-size: $font-size-lg;
  font-weight: $font-weight-semibold;
  color: $color-text-primary;
}
.sectionMore {
  font-size: $font-size-sm;
  color: $color-text-tertiary;
}
.quickGrid {
  display: flex;
  flex-wrap: wrap;
```

```
gap: $spacing-md;
margin-bottom: $spacing-lg;
}
.quickItem {
width: calc((100% - #{$spacing-md}) / 2);
background: $color-bg-card;
border-radius: $radius-lg;
padding: $spacing-md;
box-shadow: $shadow-card;
display: flex;
align-items: center;
transition: all $transition-base;
&:active {
opacity: 0.9;
transform: scale(0.98);
}
}
.quickIcon {
width: 72rpx;
height: 72rpx;
border-radius: $radius-md;
background: $color-tag-bg;
@include flex-center;
margin-right: $spacing-sm;
flex-shrink: 0;
}
.quickIconText {
font-size: 36rpx;
}
.quickInfo {
display: flex;
flex-direction: column;
min-width: 0;
}
.quickName {
font-size: $font-size-sm;
font-weight: $font-weight-medium;
color: $color-text-primary;
}
.quickDesc {
font-size: $font-size-xs;
color: $color-text-tertiary;
margin-top: 4rpx;
@include text-ellipsis;
}
// 经典名句
.classicList {
margin-bottom: $spacing-lg;
}
.classicCard {
```



```
background: $color-bg-card;
border-radius: $radius-lg;
padding: $spacing-lg;
margin-bottom: $spacing-md;
box-shadow: $shadow-card;
border-left: 6px solid $color-primary;
transition: all $transition-base;
&:active {
  opacity: 0.95;
  transform: scale(0.98);
}
}

.classicOriginal {
  display: block;
  font-size: $font-size-md;
  color: $color-text-primary;
  line-height: $line-height-loose;
  font-weight: $font-weight-medium;
  margin-bottom: $spacing-xs;
}

.classicKeyPoint {
  display: block;
  font-size: $font-size-sm;
  color: $color-text-tertiary;
}

// 搜索入口 (P2-1)
.searchBar {
  display: flex;
  align-items: center;
  background: $color-bg-card;
  border-radius: $radius-round;
  padding: 16px $spacing-lg;
  margin-bottom: $spacing-lg;
  box-shadow: $shadow-card;
  &:active {
    opacity: 0.7;
  }
}

.searchIcon {
  font-size: $font-size-md;
  color: $color-text-tertiary;
  margin-right: $spacing-sm;
  flex-shrink: 0;
}

.searchPlaceholder {
  font-size: $font-size-sm;
  color: $color-text-tertiary;
}

// 今日推荐骨架屏 (P2-5)
.dailySkeleton {
```

```
padding: $spacing-md 0;
}
/* ===== 文件: src/pages/classics/index.module.scss ===== */
@use '@styles/variables.scss' as *;
.container {
  min-height: 100vh;
  padding: 0 $page-padding $spacing-xl;
}
// 顶部简介
.introCard {
  background: linear-gradient(135deg, $color-primary 0%, $color-primary-dark 100%);
  border-radius: $radius-xl;
  padding: $spacing-lg;
  margin: $spacing-md 0 $spacing-lg;
  box-shadow: 0 8px 24px rgba(184, 97, 45, 0.2);
}
.introTitle {
  display: block;
  font-size: $font-size-xl;
  font-weight: $font-weight-semibold;
  color: $color-text-white;
  margin-bottom: $spacing-xs;
}
.introText {
  display: block;
  font-size: $font-size-sm;
  color: rgba(255, 255, 255, 0.8);
  line-height: $line-height-normal;
}
// 搜索模式切换
.modeTabs {
  display: flex;
  margin-bottom: $spacing-sm;
  background: $color-bg-card;
  border-radius: $radius-md;
  padding: 6px;
}
.modeTab {
  flex: 1;
  text-align: center;
  font-size: $font-size-sm;
  color: $color-text-secondary;
  padding: 12px 0;
  border-radius: $radius-sm;
  transition: all $transition-base;
  &.modeTabActive {
    background: $color-primary;
    color: #fff;
    font-weight: $font-weight-medium;
  }
}
```

```
}  
// 搜索栏  
.searchBar {  
  display: flex;  
  align-items: center;  
  background: $color-bg-card;  
  border-radius: $radius-round;  
  padding: 16rpx $spacing-lg;  
  margin-bottom: $spacing-md;  
  box-shadow: $shadow-card;  
}  
.searchIcon {  
  font-size: $font-size-md;  
  color: $color-text-tertiary;  
  margin-right: $spacing-sm;  
  flex-shrink: 0;  
}  
.searchInput {  
  flex: 1;  
  font-size: $font-size-sm;  
  color: $color-text-primary;  
}  
.clearBtn {  
  font-size: $font-size-md;  
  color: $color-text-tertiary;  
  padding: 8rpx;  
  flex-shrink: 0;  
}  
// 搜索结果统计  
.searchStats {  
  margin-bottom: $spacing-md;  
}  
.searchStatsText {  
  font-size: $font-size-sm;  
  color: $color-text-secondary;  
}  
// 搜索匹配章句  
.verseSection {  
  margin-bottom: $spacing-lg;  
}  
.sectionTitleRow {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  margin-bottom: $spacing-md;  
}  
.sectionTitle {  
  font-size: $font-size-md;  
  font-weight: $font-weight-semibold;  
  color: $color-primary;
```

```
}
.sectionCount {
  font-size: $font-size-sm;
  color: $color-text-tertiary;
}
.searchResultCard {
  background: $color-bg-card;
  border-radius: $radius-md;
  padding: $spacing-md;
  margin-bottom: $spacing-sm;
  box-shadow: $shadow-card;
  border-left: 6px solid $color-primary;
  transition: all $transition-base;
  &:active {
    opacity: 0.9;
    transform: scale(0.98);
  }
}
.resultHeader {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: $spacing-xs;
}
.resultChapter {
  font-size: $font-size-xs;
  color: $color-primary;
  font-weight: $font-weight-medium;
}
.resultArrow {
  font-size: $font-size-md;
  color: $color-text-tertiary;
}
// 匹配字段徽章
.matchBadges {
  display: flex;
  gap: 8px;
  flex-shrink: 0;
}
.matchBadge {
  padding: 2px 12px;
  font-size: $font-size-xs;
  border-radius: $radius-xs;
  background: $color-tag-bg;
  color: $color-primary;
}
.resultOriginal {
  display: block;
  font-size: $font-size-sm;
  color: $color-text-primary;
```

```

    line-height: $line-height-loose;
    @include text-ellipsis-multi(3);
}
// 上下文预览（译文/注释命中时展示匹配片段）
.resultPreview {
    display: block;
    font-size: $font-size-xs;
    color: $color-text-secondary;
    line-height: $line-height-loose;
    margin-top: $spacing-xs;
    padding-top: $spacing-xs;
    border-top: 1rpx solid $color-divider;
    @include text-ellipsis-multi(2);
}
// 搜索高亮文本（内联嵌套于 resultOriginal / resultPreview）
.highlight {
    color: $color-primary;
    font-weight: $font-weight-semibold;
}
.resultMore {
    display: block;
    text-align: center;
    font-size: $font-size-xs;
    color: $color-primary;
    padding: $spacing-md 0;
    &:active {
        opacity: 0.7;
    }
    padding: $spacing-sm 0;
}
// 分类标签
.tagScroll {
    margin-bottom: $spacing-md;
    white-space: nowrap;
}
.tagItem {
    display: inline-block;
    padding: 12rpx $spacing-lg;
    margin-right: $spacing-sm;
    border-radius: $radius-round;
    font-size: $font-size-sm;
    color: $color-text-secondary;
    background: $color-bg-card;
    white-space: nowrap;
    transition: all $transition-base;
    &.active {
        background: $color-primary;
        color: $color-text-white;
    }
}

```

// 篇章列表

```
.listHeader {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: $spacing-md;
}

.listTitle {
  font-size: $font-size-lg;
  font-weight: $font-weight-semibold;
  color: $color-text-primary;
}

.listCount {
  font-size: $font-size-sm;
  color: $color-text-tertiary;
}

.chapterList {
  margin-bottom: $spacing-xl;
}

.emptyTip {
  text-align: center;
  padding: $spacing-xl 0;
  font-size: $font-size-sm;
  color: $color-text-tertiary;
}

/* ===== 文件: src/packageContent/pages/verseDetail/index.module.scss ===== */
@use '@styles/variables.scss' as *;

.container {
  min-height: 100vh;
  padding: 0 $page-padding $spacing-xl;
  // 阅读字号（CSS 变量，随「设置 → 正文字号」缩放）
  --reading-size: $font-size-xl;
  --reading-secondary-size: $font-size-md;
  &.fontLarge {
    --reading-size: 42rpx;
    --reading-secondary-size: 34rpx;
  }
  &.fontXl {
    --reading-size: 48rpx;
    --reading-secondary-size: 38rpx;
  }
}

// 原文卡片
.originalCard {
  background: linear-gradient(135deg, $color-primary 0%, $color-primary-dark 100%);
  border-radius: $radius-xl;
  padding: $spacing-xl $spacing-lg;
  margin: $spacing-md 0 $spacing-lg;
  box-shadow: 0 8rpx 24rpx rgba(184, 97, 45, 0.25);
  position: relative;
```

```
overflow: hidden;
&::before {
  content: '';
  position: absolute;
  bottom: -60rpx;
  left: -40rpx;
  width: 240rpx;
  height: 240rpx;
  border-radius: 50%;
  background: rgba(255, 255, 255, 0.06);
}
}

.chapterTag {
  display: inline-block;
  font-size: $font-size-xs;
  color: rgba(255, 255, 255, 0.7);
  background: rgba(255, 255, 255, 0.15);
  padding: 4rpx 16rpx;
  border-radius: $radius-xs;
  margin-bottom: $spacing-sm;
  position: relative;
  z-index: 1;
}

.originalText {
  display: block;
  font-size: var(--reading-size);
  color: $color-text-white;
  line-height: $line-height-loose;
  font-weight: $font-weight-medium;
  margin-bottom: $spacing-md;
  position: relative;
  z-index: 1;
}

.keyPointText {
  display: block;
  font-size: $font-size-sm;
  color: rgba(255, 255, 255, 0.75);
  position: relative;
  z-index: 1;
}

// 内容区块
.sectionCard {
  background: $color-bg-card;
  border-radius: $radius-lg;
  padding: $spacing-lg;
  margin-bottom: $spacing-md;
  box-shadow: $shadow-card;
}

.sectionTitle {
  display: flex;
```

```
    align-items: center;
    margin-bottom: $spacing-sm;
}
.sectionTitleIcon {
    width: 8rpx;
    height: 32rpx;
    background: $color-primary;
    border-radius: $radius-xs;
    margin-right: $spacing-sm;
}
.sectionTitleText {
    font-size: $font-size-md;
    font-weight: $font-weight-semibold;
    color: $color-text-primary;
}
.sectionContent {
    display: block;
    font-size: var(--reading-secondary-size);
    color: $color-text-secondary;
    line-height: $line-height-loose;
    white-space: pre-wrap;
}
// 操作栏
.actionBar {
    display: flex;
    gap: $spacing-md;
    margin-bottom: $spacing-lg;
}
// 上一句 / 下一句导航
.navBar {
    display: flex;
    gap: $spacing-md;
    margin-bottom: $spacing-lg;
}
.navBtn {
    flex: 1;
    height: $button-height-md;
    border-radius: $radius-button;
    background: $color-bg-card;
    color: $color-primary;
    border: 2rpx solid $color-primary;
    box-sizing: border-box;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: $font-size-sm;
    white-space: nowrap;
    &.navBtnDisabled {
        background: $color-tag-bg;
        color: $color-text-secondary;
    }
}
```



```

        border-color: $color-tag-bg;
    }
}
.actionBtn {
    flex: 1;
    height: $button-height-md;
    border-radius: $radius-button;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: $font-size-sm;
    white-space: nowrap;
    transition: all $transition-base;
    &:active {
        opacity: 0.85;
        transform: scale(0.98);
    }
}
.actionBtnPrimary {
    background: linear-gradient(135deg, $color-primary 0%, $color-primary-light 100%);
    color: $color-text-white;
    box-shadow: 0 4px 12px rgba(184, 97, 45, 0.3);
}
.actionBtnSecondary {
    background: $color-bg-card;
    color: $color-primary;
    border: 2px solid $color-primary;
    box-sizing: border-box;
}
.actionBtnRead {
    background: $color-tag-bg;
    color: $color-text-secondary;
    &.read {
        background: rgba(91, 140, 90, 0.1);
        color: $color-success;
    }
}
// 字号调节按钮组（阅读页内 A-/A+）
.fontSizeGroup {
    display: flex;
    gap: $spacing-xs;
}
.fontSizeBtn {
    width: 88px;
    height: $button-height-md;
    border-radius: $radius-button;
    background: $color-bg-card;
    color: $color-text-secondary;
    border: 2px solid $color-divider;
    box-sizing: border-box;
}

```

```
display: flex;
align-items: center;
justify-content: center;
font-size: $font-size-sm;
transition: all $transition-base;
&:active {
  opacity: 0.85;
  transform: scale(0.96);
}
&.fontSizeBtnDisabled {
  opacity: 0.4;
}
}
// 相关心得
.insightSection {
  margin-top: $spacing-md;
}
.insightSectionTitle {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: $spacing-md;
}
.insightSectionText {
  font-size: $font-size-lg;
  font-weight: $font-weight-semibold;
  color: $color-text-primary;
}
.insightSectionCount {
  font-size: $font-size-sm;
  color: $color-text-tertiary;
}
.emptyTip {
  text-align: center;
  padding: $spacing-lg 0;
  font-size: $font-size-sm;
  color: $color-text-tertiary;
}
// 我的笔记卡片
.myNoteCard {
  background: $color-bg-card;
  border-radius: $radius-lg;
  padding: $spacing-lg;
  margin-bottom: $spacing-md;
  box-shadow: $shadow-card;
  border-left: 6px solid $color-primary;
}
.myNoteContent {
  display: block;
  font-size: $font-size-md;
```

```
    color: $color-text-primary;
    line-height: $line-height-loose;
    margin-bottom: $spacing-sm;
}

.myNoteTags {
    display: flex;
    flex-wrap: wrap;
    gap: $spacing-xs;
    margin-bottom: $spacing-sm;
}

.myNoteTag {
    display: inline-block;
    padding: 4rpx 12rpx;
    background: $color-tag-bg;
    color: $color-primary;
    font-size: $font-size-xs;
    border-radius: $radius-xs;
}

.myNoteTime {
    font-size: $font-size-xs;
    color: $color-text-tertiary;
}

.myNoteFooter {
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.myNoteActions {
    display: flex;
    gap: $spacing-md;
}

.myNoteAction {
    font-size: $font-size-xs;
    color: $color-primary;
    padding: 4rpx 16rpx;
    border-radius: $radius-xs;
    background: $color-tag-bg;
}

.myNoteActionDelete {
    font-size: $font-size-xs;
    color: $color-text-tertiary;
    padding: 4rpx 16rpx;
    border-radius: $radius-xs;
    background: rgba(153, 153, 153, 0.1);
}

/* 内容来源标注 */
.sourceBar {
    display: flex;
    justify-content: center;
    padding: $spacing-md 0 $spacing-lg;
```

[illegible]